

Thị giác 3D và hình học (3D Vision and Geometry)

PHỤ THUỘC A/02-vat-ly-cao-anh/03-camera-va-hinh-hoc-nhieu-view (mô hình chiếu, hiệu chỉnh); B/04-nen-tang-hoc-tu-du-lieu (đánh đổi giả định $\leftrightarrow$ dữ liệu); B/07-chia-du-lieu-va-danh-gia (giao thức đánh giá, ATE/RPE); C/13-kien-truc-backbone (transformer, thiên lệch quy nạp).

TÓM TẮT

Đây là lĩnh vực có mạch lịch sử rõ nhất trong computer vision: cùng một bài toán — suy hình dạng 3D và pose camera từ ảnh 2D — được giải lại từ đầu bốn lần, mỗi lần đổi một loại giả định lấy một loại độ bền. Chương này dựng khung bốn thể hệ đó, rồi đi vào các quyết định con (biểu diễn, độ sâu, SLAM, học trên point cloud) và hai thứ mới nhất là *neural rendering* và hợp nhất cảm biến. Đọc xong bạn chọn được thể hệ đúng cho dữ liệu đang có và biết cách phát hiện lúc một mô hình học đang nói dối về hình học. Nếu chỉ nhớ một điều: mỗi lần bỏ bớt một giả định về thế giới, bạn phải bù bằng dữ liệu, và bạn mất một phần khả năng tự chẩn đoán. Đó là cái giá ít được nói tới nhất của thị giác 3D hiện đại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **thị giác 3D (3D vision)** — nhánh của computer vision lo việc suy hình dạng ba chiều và chuyển động camera từ ảnh hai chiều — ngược lại với nhánh nhận dạng, vốn chỉ hỏi “trong ảnh có gì”.
- **phép chiếu phối cảnh (perspective projection)** — cách camera biến một điểm 3D thành một pixel: chia toạ độ ngang và dọc cho độ sâu. Chính phép chia này làm mất chiều thứ ba.
- **prior (prior)** — những gì mô hình “tin sẵn” về thế giới trước khi nhìn dữ liệu hiện tại — có thể do người viết cài cứng (mô hình vật lý) hoặc do mạng học từ dữ liệu lớn.
- **metric (đơn vị mét)** — một kết quả gọi là metric khi nó cho ra số đo tuyệt đối tính bằng mét, chứ không chỉ đúng sai khác một hệ số tỉ lệ chưa biết.
- **SLAM và VIO** — hai hệ thống ước lượng vị trí robot theo thời gian thực trong lúc dựng bản đồ, dùng camera (SLAM) hoặc camera cộng cảm biến quán tính (VIO). Chương này giả sử bạn đã quen chúng và chỉ đặt chúng vào bức tranh chung.
- **neural rendering** — họ phương pháp học một biểu diễn cảnh bằng cách bắt nó render ra ảnh giống các ảnh đã chụp, thay vì bằng cách đo hình học trực tiếp.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao thêm bao nhiêu dữ liệu cũng không cho một camera đơn biết vật thể trước mặt lớn thật sự bao nhiêu mét?
- Q2. Nếu một *transformer* suy được pose và hình học chỉ trong một lượt xuôi, vì sao COLMAP vẫn chưa bị thay thế?
- Q3. Vì sao NeRF phải mô tả cảnh bằng khói mờ thay vì bằng bề mặt cứng, trong khi thứ tồn tại thật là bề mặt?
- Q4. Vì sao một hệ hợp nhất camera–LiDAR có thể chạy hoàn hảo lúc robot đứng yên rồi hỏng đúng lúc nó xoay nhanh, dù không một tham số nào thay đổi?

Vì sao chương này chia làm ba mục

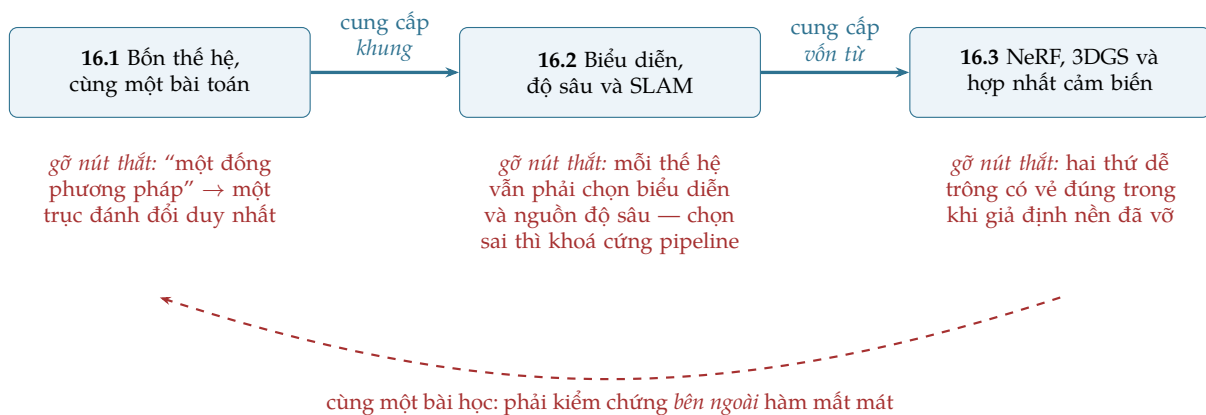
Chương được chia theo ba tầng câu hỏi, và thứ tự là thứ tự phụ thuộc chứ không phải thứ tự lịch sử.

Mục thứ nhất dựng *khung*: bốn thể hệ giải cùng một bài toán, mỗi thể hệ thay một đoạn khác nhau của cùng một pipeline. Không có khung này thì mọi thứ còn lại rơi vào tình trạng “một đồng phương pháp”, và ta không trả lời được câu hỏi thực dụng nhất — với dữ liệu đang có thì nên dùng cái gì. Khung này cũng là chỗ trực đánh đổi “giả định mạnh + ít dữ liệu $\leftrightarrow$ giả định yếu + nhiều dữ liệu” của chương 4 hiện ra rõ nhất trong toàn bộ CV.

Mục thứ hai đi vào các *quyết định con* mà mọi thể hệ đều phải đưa ra: lưu cảnh bằng biểu diễn nào, lấy độ sâu từ đâu và cái gì cố định tỉ lệ, SLAM đứng ở đâu trong bức tranh, và làm sao đưa mạng nơ-ron lên một tập điểm không thứ tự. Đây là tầng “vốn từ”: không nắm nó thì không đọc được mục ba.

Mục thứ ba đào sâu đúng hai thứ xa nền tảng hình học cổ điển nhất — neural rendering (NeRF, 3DGS) và hợp nhất camera–LiDAR–IMU. Hai chủ đề này được gộp vào một mục vì chúng chia chung một bài học: cả hai đều dễ trông có vẻ hoạt động trong khi giả định nền đã bị vi phạm, và cả hai đều đòi một cơ chế kiểm chứng *bên ngoài* hàm mất mát của chính chúng.

Hình 1.1 vẽ ba mục đó cùng nút thắt mà mỗi mục gỡ.



Hình 1.1. Ba mục của chương và quan hệ phụ thuộc giữa chúng. Thứ tự là thứ tự *phụ thuộc*, không phải thứ tự lịch sử: mục 16.1 dựng khung để hai mục sau không biến thành danh sách phương pháp rời rạc, mục 16.2 cung cấp vốn từ cho mục 16.3. Mũi tên đứt phía dưới là điều mà cả ba mục cùng dẫn tới, và cũng là điều dễ bị bỏ sót nhất khi đọc riêng từng mục.

Với người đọc đang làm visual-inertial SLAM thật, ba mục này có mật độ thông tin mới rất khác nhau: mục một chủ yếu sắp xếp lại thứ đã biết, mục hai bổ sung phần học sâu trên dữ liệu 3D, còn mục ba mới là phần thực sự mới. Nếu thời gian eo hẹp, đọc mục một để lấy khung rồi nhảy thẳng sang mục ba.

Bản đồ chương 16 — Thị giác 3D và hình học

TÓM TẮT

Chương 16 gồm ba mục, chia theo ba tầng câu hỏi chứ không theo thứ tự lịch sử: khung bốn thể hệ, các quyết định con, rồi hai chủ đề mới nhất. Bản đồ này nói mục nào là cốt lõi, mục nào có thể tra khi cần, và đọc theo trình tự nào tùy theo nền sẵn có của bạn. Nếu chỉ nhớ một điều: mục 16.1 là khung để đọc hai mục còn lại — bỏ qua nó thì phần sau biến thành một danh sách phương pháp rời rạc.

KIẾN THỨC CẦN CÓ TRƯỚC

- **SfM / MVS** — hai bước của pipeline hình học cổ điển: suy pose camera và cấu trúc thưa từ ảnh (SfM), rồi dựng bề mặt dày đặc khi đã biết pose (MVS).
- **neural rendering** — học một biểu diễn cảnh bằng cách bắt nó dựng ra ảnh giống các ảnh đã chụp, thay vì đo hình học trực tiếp.
- **feed-forward geometry** — mô hình suy pose và hình học trong *một* lần chạy mạng, không cần vòng tối ưu và không cần pose cho trước.
- **point cloud / voxel / mesh** — ba cách lưu hình dạng 3D: tập điểm rời rạc, lưới ô đều, và bề mặt tam giác.

Ba mục và vai trò của từng mục

- **16.1 — Bốn thể hệ, cùng một bài toán (cốt lõi).** Khung của cả chương: hình học cổ điển, lai học-hình học, neural rendering, feed-forward geometry. Trục đánh đổi “giả định mạnh + ít dữ liệu ↔ giả định yếu + nhiều dữ liệu”. Đọc mục này trước mọi thứ khác.
- **16.2 — Chọn biểu diễn 3D, độ sâu và SLAM (cốt lõi phần biểu diễn và độ sâu; phần SLAM là tham khảo với người đã làm SLAM).** Vốn từ: voxel, point cloud, mesh, trường ẩn, Gaussian; bốn nguồn độ sâu và cái gì cố định tỉ lệ; PointNet và bất biến hoán vị.
- **16.3 — NeRF, 3DGS và hợp nhất cảm biến (cốt lõi).** Cơ chế dựng ảnh thể tích, vì sao phải dùng mật độ liên tục, 3DGS đổi gì lấy gì, ba giả định nền dễ vỡ; early đối lại late fusion và ngân sách sai số hiệu chỉnh.

Trình tự đọc

1. Nếu bạn **chưa có nền hình học nhiều view**: đọc mục *camera và hình học nhiều view* ở A/02-vat-ly-tao-anh trước, rồi 16.1 → 16.2 → 16.3 theo đúng thứ tự.
2. Nếu bạn **đã làm SLAM hoặc SfM**: đọc 16.1 để lấy khung (nhẹ, phần lớn là sắp xếp lại thứ đã biết), lướt mục 16.2 phần D về SLAM, đọc kỹ 16.2 phần C và E (độ sâu học được, point cloud) và toàn bộ 16.3 — đó là phần có mật độ thông tin mới cao nhất với bạn.
3. Khối *Thực hành* nằm ở file chương, sau ba mục.

Chương này phụ thuộc gì và được dùng lại ở đâu

- **Phụ thuộc lên trên:** A/02-vat-ly-tao-anh (mô hình chiếu, hiệu chỉnh, phơi sáng); B/04-nen-tang-hoc-tu-du-lieu (đánh đổi giả định ↔ dữ liệu, tính xác định được); B/07-chia-du-lieu-va-danh-gia (ATE/RPE, giao thức); C/13-kien-truc-backbone (transformer).
- **Được dùng lại ở dưới:** C/17-mo-hinh-sinh/02-toi-uu-tren-pixel dùng lại nguyên ý tưởng tối ưu trên chính đối tượng cần tạo ra; C/17-mo-hinh-sinh/04-low-level-vision dùng lại

khái niệm bài toán ill-posed và vai trò của prior; E/25-do-tin-cay-va-van-hanh dùng lại yêu cầu “cơ chế phát hiện sai từ bên ngoài”.

1.1 Bốn thế hệ, cùng một bài toán (Four Generations of 3D Reconstruction)

PHỤ THUỘC A/02-vat-ly-tao-anh/03-camera-va-hinh-hoc-nhieu-view (mô hình chiếu, ràng buộc epipolar); B/04-nen-tang-hoc-tu-du-lieu (trục “giả định mạnh + ít dữ liệu” ↔ “giả định yếu + nhiều dữ liệu”); B/03-giai-phau-he-thong (tách bài toán → nguyên lý → cài đặt).

TÓM TẮT

Bài toán khôi phục hình học 3D từ ảnh đã được giải lại từ đầu bốn lần. Bốn thế hệ đó không thay thế nhau: mỗi thế hệ thay một *đoạn khác nhau* của cùng một pipeline. Vì vậy chúng chồng lớp được, và câu hỏi “VGGT có thay được COLMAP không” là câu hỏi sai. Đọc xong bạn làm được ba việc. Chọn thế hệ đúng cho dữ liệu đang có: bao nhiêu ảnh, có pose hay không, cảnh loại gì. Ghép hai thế hệ thành một pipeline, một bên khởi tạo một bên tinh chỉnh. Và kiểm tra được lúc một mô hình tiến thẳng đang nói dối. Nếu chỉ nhớ một điều: đi từ thế hệ một tới thế hệ bốn, thứ mất đi không chỉ là độ chính xác metric mà là *khả năng tự chẩn đoán*. Hiệu chỉnh chùm tia trả về phần dư và hiệp phương sai; mạng tiến thẳng không trả về gì cả. Vì vậy đưa nó vào hệ thống thật là phải tự dựng lại cơ chế phát hiện sai từ bên ngoài.

KIẾN THỨC CẦN CÓ TRƯỚC

- **SfM (structure from motion)** — quy trình cổ điển suy đồng thời hình dạng 3D của cảnh và vị trí camera từ một tập ảnh chụp cùng cảnh ở nhiều góc.
- **MVS (multi-view stereo)** — bước sau SfM: khi đã biết pose camera, dựng bề mặt dày đặc bằng cách khớp từng vùng ảnh giữa nhiều view.
- **pose camera (camera pose)** — vị trí và hướng của camera trong không gian, sáu bậc tự do; “biết pose” là biết ảnh được chụp từ đâu, nhìn về đâu.
- **tam giác hoá (triangulation)** — tìm điểm 3D bằng giao của hai hay nhiều tia nhìn xuất phát từ các camera đã biết pose; đây là chỗ chiều bị mất khi chiếu được lấy lại.
- **hiệu chỉnh chùm tia (bundle adjustment)** — bài toán bình phương tối thiểu phi tuyến. Nó tinh chỉnh đồng thời mọi pose và mọi điểm 3D sao cho sai số chiếu lại nhỏ nhất. Bạn đã quen nó; điều cần nhớ ở đây là nó trả về *phần dư* và *hiệp phương sai*.
- **RANSAC** — thủ tục lọc điểm sai bằng cách thử ngẫu nhiên nhiều giả thuyết nhỏ rồi giữ giả thuyết được nhiều dữ liệu ủng hộ nhất. Nó giả định điểm sai là *ngẫu nhiên*, không phải sai đồng loạt theo một kiểu.
- **point map** — ảnh mà mỗi pixel chứa một toạ độ 3D thay vì một màu; đây là dạng đầu ra của các mô hình tiến thẳng hiện đại, thay cho “point cloud rời rạc”.
- **disparity** — độ lệch ngang của cùng một điểm giữa ảnh trái và ảnh phải trong hệ stereo; nó tỉ lệ nghịch với độ sâu.

A. Đặt vấn đề

Bài toán gốc của thị giác 3D không đổi trong bốn mươi năm: cho một tập ảnh 2D của cùng một cảnh, hãy khôi phục hình dạng ba chiều của cảnh và vị trí camera lúc chụp. Nó khó vì phép chiếu phối cảnh làm mất một chiều — mỗi pixel chỉ nói “có gì đó nằm đâu đó trên tia này”, chứ không nói nó xa bao nhiêu. Chỉ khi nhiều tia từ nhiều vị trí camera khác nhau cắt nhau thì độ sâu mới được xác định, và cả ngành 3D là các cách khác nhau để lấy lại thông tin đã mất đó.

Điều đáng học ở lĩnh vực này không phải thuật toán cụ thể mà là mạch lịch sử của nó: đây là chỗ duy nhất trong computer vision mà ta thấy một bài toán được giải lại từ đầu bốn lần, mỗi lần đổi một loại giả định lấy một loại độ bền. Thế hệ đầu viết thẳng mô hình vật lý vào bài toán tối ưu, nên chỉ cần vài chục ảnh nhưng đòi cảnh phải cư xử đúng như mô hình. Thế hệ cuối vứt gần hết mô hình đó đi. Nó thay bằng một *prior* học từ hàng triệu cảnh, nên chạy được ở chỗ mô hình vật lý bó tay. Cái giá là không còn cách nào kiểm chứng kết quả từ bên trong. Đây chính xác là trục “giả định mạnh + ít dữ liệu $\leftrightarrow$ giả định yếu + nhiều dữ liệu” của chương 4, và không chỗ nào trong CV nó hiện ra sạch sẽ như ở đây.

$\leftrightarrow$ **trường hợp riêng của B/04** — bốn thế hệ 3D là hiện thân cụ thể nhất của trục “giả định mạnh + ít dữ liệu $\leftrightarrow$ giả định yếu + nhiều dữ liệu”; ở đây ta đo được cái giá bằng centimét.

Học mục này thì làm được gì mà trước đó không làm được? Trả lời được ba câu hỏi thực dụng: (i) với dữ liệu bạn đang có — bao nhiêu ảnh, có pose hay không, cảnh loại gì — thế hệ nào là lựa chọn đúng; (ii) khi ghép hai thế hệ vào một pipeline thì ai làm khởi tạo, ai làm tinh chỉnh; (iii) làm thế nào phát hiện lúc một mô hình **tiền thẳng** (*feed-forward*) đang nói dối, việc mà **hiệu chỉnh chùm tia** (*bundle adjustment*) làm hộ bạn miễn phí còn mạng học thì không.

Với người đọc đang làm *visual-inertial SLAM*, ba thế hệ đầu phần lớn là sân nhà. Phần lạ nằm ở hai thế hệ sau — *neural rendering* và *feed-forward geometry* — nên chương này viết kỹ đúng hai chỗ đó.

TRỰC GIÁC

Bốn thế hệ là bốn cách trả lời câu hỏi “căn phòng này rộng bao nhiêu”. Thế hệ hình học mang thước ra đo từng cạnh rồi cộng sai số theo quy tắc rõ ràng: đúng tới milimét, và nếu các số đo mâu thuẫn nhau thì phần dư của phép bình phương tối thiểu tự nó tố cáo. Thế hệ lai vẫn dùng thước, nhưng nhờ mắt chỉ chỗ đặt thước cho nhanh. *Neural rendering* không đo phòng: nó dựng một mô hình phòng bằng sương mù có màu rồi chỉnh đám sương cho tới khi ảnh chụp mô hình trùng khít mọi ảnh thật — đúng về mặt *nhìn*, chưa chắc đúng về mặt *chạm*. Thế hệ tiền thẳng liếc một cái rồi nói “khoảng bốn nhân năm mét”, vì đã ở trong hàng vạn căn phòng: nhanh, thường đúng, và khi sai thì sai rất tự tin vì không có phép đo nào để mâu thuẫn với nó.

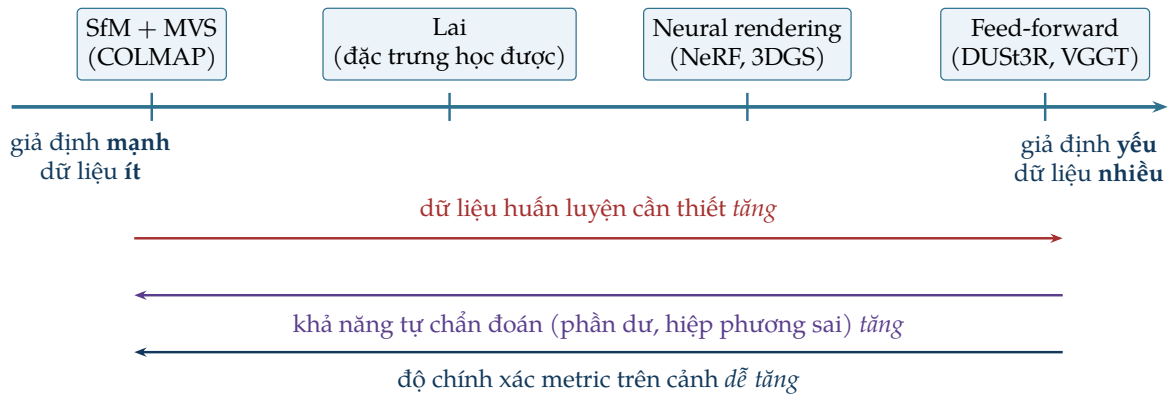
B. Bốn thế hệ đặt cạnh nhau

Bảng 1.1 đặt bốn thế hệ trên cùng bốn tiêu chí. Cột đáng đọc kỹ nhất là cột cuối: mỗi thế hệ ra đời vì thế hệ trước hỏng ở một chỗ cụ thể, rồi lại hỏng ở một chỗ khác.

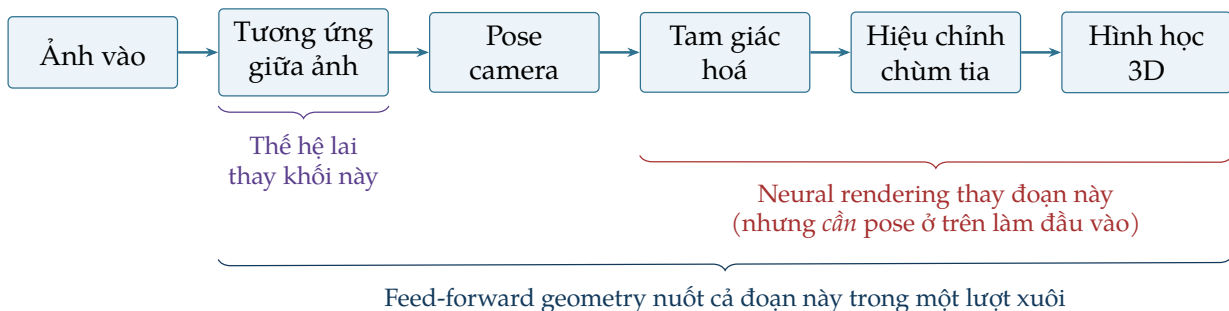
Bảng 1.1. Bốn thể hệ giải bài toán khôi phục hình học 3D từ ảnh. Không thể hệ nào thay thế hoàn toàn thể hệ trước; trên thực tế chúng chồng lên nhau.

Thể hệ	Cơ chế	Giả định	Mạnh khi	Hồng khi
Hình học cổ điển (SfM/MVS, COLMAP)	Tìm tương ứng, ước lượng pose, tam giác hoá, hiệu chỉnh chùm tia	Cảnh tĩnh, đủ texture, đủ chồng lấn giữa các view	Cần độ chính xác metric và cần kiểm chứng được	Ít texture, ít ảnh, bề mặt phản chiếu hoặc trong suốt
Lai học + hình học	Học phần tương ứng hoặc phần độ sâu, giữ nguyên phần tối ưu hình học	Hình học vẫn đúng; chỉ khâu nhận dạng cần học	Có prior mạnh nhưng vẫn cần ràng buộc cứng	Khi khâu học sai một cách <i>có hệ thống</i> : RANSAC không lọc nổi
Neural rendering (NeRF, 3DGS)	Tối ưu một biểu diễn riêng cho từng cảnh bằng loss render khả vi	Đã biết pose; cảnh tĩnh; phơi sáng nhất quán	Cần chất lượng ảnh ở góc nhìn mới	Không có pose sẵn; phải tối ưu lại từ đầu cho mỗi cảnh
Feed-forward geometry (DUST3R, VGGT)	Một <i>transformer</i> suy ra pose + độ sâu + point map trong một lượt xuôi	Prior học từ dữ liệu lớn khái quát được sang cảnh mới	Ít ảnh, không có pose, cần kết quả nhanh	Cảnh dễ mà cần độ chính xác metric: thua hiệu chỉnh chùm tia

Bảng cho biết *cái gì*, hai hình sau cho biết *tại sao*. Hình 1.2 đặt bốn thể hệ lên một trục duy nhất và cho thấy ba đại lượng cùng biến thiên theo trục đó; Hình 1.3 cho thấy mỗi thể hệ thay một *đoạn* khác nhau của cùng một pipeline. Hình thứ hai giải thích vì sao câu hỏi “VGGT có thay được COLMAP không” là câu hỏi sai — câu hỏi đúng là “thay được đoạn nào”.



Hình 1.2. Một trục duy nhất giải thích cả bốn thể hệ. Di sang phải, ta bỏ bớt giả định về thể giới và bù bằng dữ liệu; đổi lại, hai thứ cùng giảm — độ chính xác metric ở cảnh dễ, và quan trọng hơn, khả năng hệ thống tự biết mình đang sai.



Hình 1.3. Cùng một pipeline, ba cách thay thế khác nhau. Thể hệ lai thay đúng khối dễ hỏng nhất mà giữ nguyên phần tối ưu; neural rendering nằm *sau* khâu pose nên nó là khách hàng của SfM chứ không phải đối thủ; chỉ thể hệ tiến thẳng mới thực sự cạnh tranh trực tiếp với COLMAP.

C. Vì sao thế hệ sau ra đời

I Mạch 1 — bốn thế hệ, cùng một bài toán

- 1 Vấn đề** ảnh mất một chiều, và không có cách nào lấy lại nó từ một ảnh đơn lẻ.
- Giải pháp** viết thẳng mô hình chiếu vào một bài toán bình phương tối thiểu: tìm tương ứng giữa các ảnh, ước lượng pose tương đối, tam giác hoá điểm, rồi tinh chỉnh toàn bộ bằng hiệu chỉnh chùm tia [1], [2].
- Giả định** cảnh tĩnh, bề mặt *Lambertian* đủ texture, nhiễu đo là Gauss độc lập.
- Hệ quả** mô hình sinh ảnh đã được cài cứng, nên phương pháp cần rất ít dữ liệu: vài chục ảnh là ra kết quả metric. Quan trọng hơn, nó trả về *phần dư* cùng *ma trận hiệp phương sai*, tức là tự nói cho ta biết nó tin kết quả tới đâu.
- 2 Vấn đề** khâu yếu nhất của thế hệ một là tìm tương ứng: nó dựa vào *descriptor* thủ công, và descriptor thủ công chết ở thay đổi ánh sáng lớn, góc nhìn lệch nhiều, vùng hoa văn lặp lại.
- Giải pháp** thay đúng khối đó bằng một mạng học — detector/descriptor học được, matcher học được, hoặc **khớp dày đặc** (*dense matching*) — và giữ nguyên phần tối ưu hình học phía sau.
- Giả định** hình học vẫn đúng; chỉ phần “nhận dạng” mới cần học.
- Giá phải trả** mất tính tái lập bit-for-bit và mất khả năng suy luận giải tích về sai số của khâu đầu.
- Vấn đề mới** mạng học sai theo kiểu *tương quan*. Ở một mặt tường lặp hoa văn, nó không cho ra *outlier* ngẫu nhiên mà cho ra một tập tương ứng sai *nhất quán*. RANSAC không lọc được loại lỗi đó, vì nó giả định outlier là ngẫu nhiên.
- ↔ **hệ quả của** B/08 — sai đồng loạt của khâu học là lệch phân phối biểu hiện ở tầng hình học; đó là lý do một bộ lọc thống kê không cứu được nó.
- 3 Vấn đề** ngay cả khi hình học đúng, đầu ra của SfM/MVS là point cloud hoặc mesh — render lại thì thùng lỗ, không có hiệu ứng phụ thuộc góc nhìn.
- Giải pháp** bỏ luôn ý định khôi phục bề mặt; tham số hoá cảnh bằng một trường liên tục và tối ưu trường đó sao cho ảnh render khớp mọi ảnh quan sát [3].
- Giả định** pose đã biết trước, cảnh tĩnh, phơi sáng và cân bằng trắng nhất quán.
- Hệ quả** chất lượng ảnh ở góc nhìn mới nhảy vọt.
- Giá phải trả** hàm mất mát chỉ ràng buộc *ảnh render*, không ràng buộc *hình học*.
- Vấn đề mới** mỗi cảnh phải tối ưu lại từ đầu, và pose vẫn phải xin từ thế hệ một.
- 4 Vấn đề** cả ba thế hệ trên đều gậy ở cùng một điều kiện: quá ít ảnh, hoặc chồng lấn không đủ để tam giác hoá. Ba tấm ảnh chụp ba góc một căn phòng, mỗi cặp chồng lấn 20 %, thì COLMAP thường không khởi tạo nổi.
- Giải pháp** huấn luyện một transformer trên rất nhiều cảnh có chân trị 3D, cho nó nhận thẳng một tập ảnh không pose và xuất ra *point map* trong một hệ toạ độ chung [4]; dòng kế tiếp (DUST3R → MAST3R → VGGT) mở rộng từ hai ảnh lên hàng chục ảnh trong một lượt xuôi.
- Giả định** cảnh mới nằm trong phân phối mà prior đã học.
- Hệ quả** bài toán đổi bản chất: từ ước lượng có ràng buộc cứng thành hồi quy.
- Giá phải trả** độ chính xác metric ở cảnh dễ thua hiệu chỉnh chùm tia, và — điểm quan trọng nhất — không còn phần dư hình học để phát hiện lúc nó sai.

Đọc theo mạch này thì thấy cả bốn bước là cùng một phép đánh đổi lặp lại: mỗi lần bỏ bớt một giả định về thế giới, ta phải bù bằng dữ liệu, và mất đi một phần khả năng tự kiểm tra — đúng ba mũi tên của Hình 1.2.

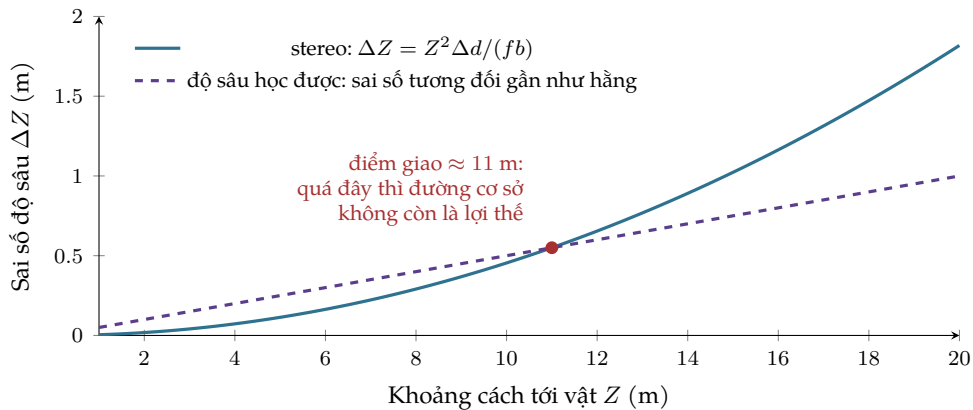
D. Một phép tính nhỏ: vì sao “chính xác metric” là đặc sản của thế hệ một

Nói gọn thì: sai số độ sâu của phương pháp hình học có một *công thức*, còn sai số của mạng học thì không. Với một cặp *stereo* đã hiệu chỉnh, độ sâu suy ra từ **độ lệch ngang** (*disparity*) theo $Z = fb/d$, nên

$$\Delta Z = \frac{Z^2}{fb} \Delta d.$$

Câu này nói: sai khớp điểm *một chút* ở gần thì hầu như không sao, nhưng cùng sai số đó ở xa thì bị khuếch đại theo bình phương khoảng cách. Lấy con số của một bộ EuRoC quen thuộc, $f \approx 400$ pixel và $b \approx 0,11$ m: ở $Z = 10$ m thì $d = 4,4$ pixel, và $\Delta d = 0,2$ pixel cho $\Delta Z \approx 0,45$ m (4,5%); ở $Z = 2$ m thì cùng Δd đó chỉ cho $\Delta Z \approx 1,8$ cm.

Hình 1.4 vẽ hai đường sai số đó cạnh nhau và cho thấy chúng cắt nhau ở đâu.



Hình 1.4. Hai loại sai số độ sâu có *hình dạng* khác nhau, và đó mới là điều đáng nhớ. Sai số stereo tăng theo Z^2 nên rất tốt ở gần và vô dụng ở xa; sai số của một mạng học được tăng gần như tuyến tính vì nó là sai số *tương đối* do prior quyết định, không do hình học. Vì vậy tồn tại một khoảng cách mà quá nó, phép đo hình học thua phép đoán có học. *Con số minh họa* ($f = 400$ px, $b = 0,11$ m, $\Delta d = 0,2$ px, sai số tương đối 5%), không phải số đo trên một hệ cụ thể.

Ba hệ quả rút ra:

- **Có một “chân trời” độ sâu.** Vượt qua nó, độ sâu stereo vô dụng — không phải vì thuật toán kém mà vì hình học.
- **Cách duy nhất đẩy chân trời ra xa là tăng fb .** Bundle adjustment trên một chuỗi ảnh chính là cách tăng **đường cơ sở** (*baseline*) hiệu dụng bằng thời gian; đây là lý do SLAM đơn mắt vẫn có độ sâu tốt khi robot đi đủ xa.
- **Mạng suy độ sâu từ một ảnh không có phương trình này.** Sai số của nó do prior quyết định, nên không tuân theo Z^2 và cũng không hưởng lợi khi ta đi thêm một quãng để mở rộng đường cơ sở. Trộn hai loại sai số này vào cùng một bộ lọc mà không mô hình hoá đúng thì hiệp phương sai sẽ sai.

E. Giới hạn và trường hợp hồng

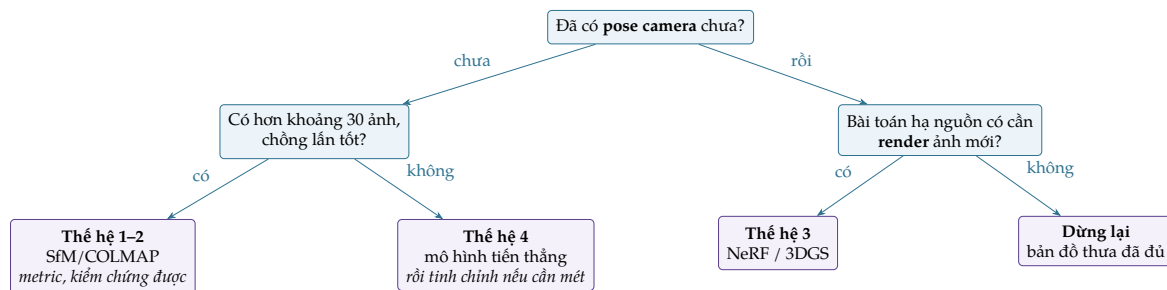
Giới hạn lớn nhất của cách kể “bốn thế hệ” là nó gợi ý một sự thay thế tuần tự, trong khi thực tế là chồng lớp. Pipeline mạnh nhất hiện nay thường lai, gồm ba chặng. Thứ nhất, dùng feed-forward geometry để có khởi tạo pose và point map thô, không cần khởi tạo thủ công. Thứ hai, ném khởi

tạo đó vào hiệu chỉnh chùm tia cổ điển để lấy lại độ chính xác metric. Thứ ba, dựng biểu diễn render lên trên. Thế hệ bốn giải bài toán *khởi tạo* (vốn là chỗ SfM hay chết), thế hệ một giải bài toán *tinh chỉnh*.

Các chế độ hỏng riêng:

- **Thế hệ một** hỏng khi giả định vật lý bị vi phạm — mặt kính, mặt nước, tường trắng, cảnh động — và nó hỏng *ôn ào*: số *inlier* tụt, phần dư tăng, hệ thống biết mình đang hỏng.
- **Thế hệ hai** kế thừa toàn bộ chỗ hỏng đó và thêm **lệch phân phối** (*domain shift*) của khâu học.
- **Thế hệ ba** hỏng ở ba chỗ: pose đầu vào sai, phơi sáng tự động thay đổi giữa các ảnh, hoặc cảnh có vật chuyển động. Riêng pose thì chỉ cần sai vài phần mười độ: ảnh render mờ đi, và tối ưu bù lại bằng vật chất giả lơ lửng.
- **Thế hệ bốn** hỏng khi cảnh ra khỏi phân phối huấn luyện — và đây là chế độ nguy hiểm nhất vì nó *im lặng*.

Gộp lại, bốn chế độ hỏng đó cho một quy tắc chọn dùng được ngay: Hình 1.5 hỏi theo đúng thứ tự các ràng buộc, từ ràng buộc cứng nhất trở đi.



Hình 1.5. Chọn thế hệ nào, hỏi theo đúng thứ tự. Câu hỏi đầu tiên không phải “phương pháp nào mới nhất” mà “tôi đã có pose chưa” — vì neural rendering nằm sau khâu pose nên nó không bao giờ là câu trả lời khi chưa có pose. Nhánh phải dưới cùng là nhánh hay bị bỏ quên nhất: rất nhiều bài toán robot chỉ cần định vị, và ở đó bản đồ thừa cổ điển vẫn là lựa chọn đúng.

CẠM BẮY

Cạm bẫy phổ biến nhất khi so sánh thế hệ bốn với COLMAP là so trên chính loại cảnh mà thế hệ bốn được huấn luyện. Nhiều bộ *benchmark* 3D lớn đã được dùng làm dữ liệu huấn luyện cho các mô hình tiến thẳng gần đây, nên con số “tốt hơn COLMAP” trên các bộ đó không nói được gì về cảnh của bạn. Phép thử trung thực duy nhất là chạy trên dữ liệu bạn tự quay. Và cách phát hiện lỗi duy nhất là kiểm tra chéo bằng một ràng buộc bên ngoài. Ba ràng buộc rẻ nhất: chiều dài một vật đã biết, một đoạn quỹ đạo có chân trị, hoặc phần dư sau khi ép kết quả tiến thẳng qua một vòng hiệu chỉnh chùm tia.

↪ **cùng nguyên lý với** C/17/04 — khôi phục ảnh cũng đối ràng buộc cứng lấy prior học được, và cũng mất khả năng tự phát hiện sai theo đúng cách này.

↪ **tiền đề** A/02 — mọi phép tam giác hoá đứng trên mô hình chiếu và giả định hiệu chỉnh; sai ở tầng đó thì không thể hệ nào cứu được.

CẦN NHỚ

Đi từ thế hệ một tới thế hệ bốn, thứ mất đi không chỉ là độ chính xác metric mà là *khả năng tự chẩn đoán*. Hiệu chỉnh chùm tia trả về phần dư và hiệp phương sai, nên hệ thống biết lúc nào nó không tin được; một mạng tiến thẳng trả về point map với cùng vẽ tự tin dù cảnh

nằm trong hay ngoài phân phối. Khi đưa thể hệ bốn vào hệ thống thật, việc đầu tiên phải làm là dựng lại một cơ chế phát hiện sai từ bên ngoài. [T3]

F. Kết nối

Mục này là khung để đọc hai mục còn lại của chương. Mục kế tiếp, [C/16-thi-giac-3d/02-bieu-dien-3d-va-depth](#), đi vào các quyết định con bên trong mỗi thể hệ; mục cuối, [C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat](#), đào sâu đúng hai thứ lạ nhất với người làm SLAM.

Trục đánh đổi ở đây là trường hợp riêng, cụ thể nhất, của B/04-nen-tang-hoc-tu-du-lieu. Cơ chế “prior thay ràng buộc cứng” sẽ trở lại nguyên vẹn dưới tên bài toán *ill-posed* ở C/17-mo-hinh-sinh/04-low-level-vision. Mô hình chiếu và các giả định hiệu chỉnh nằm ở A/02-vat-ly-tao-anh/03-camera-va-hinh-hoc-nhieu-view; còn nếu khâu tương ứng hỏng vì domain shift thì B/08-dich-chuyen-phan-phoi là chỗ đọc tiếp.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao thể hệ lai “chỉ học khâu tương ứng” lại nguy hiểm hơn thể hệ cổ điển ở đúng một loại cảnh, dù RANSAC vẫn được giữ nguyên?
2. Neural rendering đúng *sau* khâu pose trong pipeline. Điều đó nói gì về câu “NeRF thay thế SfM”?
3. Một mạng tiến thẳng cho ra point map đẹp trên video bạn quay. Bạn có ba mươi phút để kiểm tra xem nó đúng hay chỉ trông đúng — bạn làm gì?
4. Sai số độ sâu stereo tăng theo Z^2 . Vì sao sai số của độ sâu đơn ảnh học được lại *không* tuân theo quy luật đó, và điều này ảnh hưởng thế nào tới việc gán trọng số cho nó trong một bộ lọc?
5. Nếu phải chọn đúng một thể hệ để đưa vào robot chạy 24/7 ngoài trời, bạn chọn thể hệ nào và dựng thêm cơ chế an toàn nào?

Nguồn

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [2] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [3] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [4] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [5] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [6] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [7] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [8] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [9] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [10] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [11] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [12] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [13] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [14] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [15] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.

1.2 Chọn biểu diễn 3D, độ sâu và SLAM (3D Representations, Depth and SLAM)

PHỤ THUỘC C/16-thi-giac-3d/01-bon-the-he-3d (bốn thể hệ); B/10-thiet-ke-kien-truc (thiết kế theo bất biến); B/07-chia-du-lieu-va-danh-gia (ATE/RPE và giao thức đánh giá); A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (tính xác định được của tham số).

TÓM TẮT

Mục này nói về ba quyết định con nằm bên trong mọi hệ thống 3D: lưu cảnh bằng biểu diễn nào, lấy độ sâu từ đâu, và học sâu làm gì được trên dữ liệu không phải lưới. Đọc xong bạn làm được ba việc. Chọn biểu diễn cho một pipeline mới mà không phải viết lại nó sáu tháng sau. Nhìn một mô-đun độ sâu là biết nó có quyền xuất hiện trong phần dư metric hay không. Và đọc một mạng trên point cloud để biết nó mù với cái gì. Nếu chỉ nhớ hai điều: biểu diễn quyết định *phép toán nào rẻ*, nên phải chọn nó theo khâu cuối cùng của pipeline chứ không theo khâu đầu tiên. Và độ sâu đơn ảnh không phải phép đo mà là suy đoán có học, với một tỉ lệ trôi giữa các khung hình — đó chính là chỗ nó phá hỏng một bộ ước lượng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **voxel** — ô lập phương trong một lưới đều 3D — “pixel của không gian ba chiều”. Đơn giản nhưng số ô tăng theo lũy thừa ba của độ phân giải.
- **point cloud (point cloud)** — một tập tọa độ 3D không có thứ tự và không có kết nối giữa các điểm; đây là dạng dữ liệu thô của LiDAR và của stereo dày đặc.
- **mesh (mesh)** — biểu diễn bề mặt bằng các đỉnh nối thành tam giác; chuẩn của đồ họa máy tính, vì nó cho biết *đâu là mặt* chứ không chỉ *đâu có vật chất*.
- **SDF (signed distance function)** — một hàm gán cho mỗi điểm trong không gian khoảng cách có dấu tới bề mặt gần nhất — âm ở trong, dương ở ngoài. Bề mặt chính là tập điểm có giá trị 0, nên nó được định nghĩa rất sạch.
- **trường ẩn (implicit field)** — cách mô tả hình dạng bằng một *hàm* (thường là một mạng nhỏ) thay vì bằng một danh sách phần tử; SDF là ví dụ điển hình.
- **đường cơ sở (baseline)** — khoảng cách giữa hai tâm camera trong một hệ stereo; nó là thứ duy nhất cấp đơn vị mét cho phép đo độ sâu.
- **ATE và RPE** — hai chỉ số đánh giá quỹ đạo: ATE đo sai lệch tuyệt đối sau khi căn chỉnh toàn cục hai quỹ đạo, RPE đo sai lệch trên các đoạn ngắn nên không cần căn chỉnh. Bạn đã quen hai chỉ số này; ở đây chỉ dùng chúng làm ví dụ về việc “phép căn chỉnh che giấu điều gì”.
- **bất biến hoán vị (permutation invariance)** — tính chất của một hàm mà kết quả không đổi khi ta xáo trộn thứ tự các phần tử đầu vào; bắt buộc phải có nếu đầu vào là một *tập*.

A. Đặt vấn đề

Trong ảnh 2D không ai phải chọn biểu diễn: ảnh là một lưới pixel, hết chuyện. Trong 3D thì không có biểu diễn mặc định nào cả, và lựa chọn đó quyết định nhiều hơn cả việc chọn kiến trúc mạng. Lý do rất cụ thể: **biểu diễn quyết định phép toán nào rẻ**. Trên lưới *voxel*, tích chập là rẻ nhưng bộ nhớ tăng theo lũy thừa ba. Trên *point cloud*, bộ nhớ rẻ nhưng “lân cận” không có sẵn nên mọi phép cục bộ phải trả tiền tìm kiếm. Trên *mesh*, bề mặt là tường minh nên render và mô phỏng vật lý rẻ, nhưng thay đổi *topology* thì gần như không khả vi. Một khi đã chọn biểu diễn, bạn đã chọn luôn tập bài toán còn giải được hiệu quả — và đây là quyết định khó đảo ngược nhất trong một pipeline 3D.

Câu chuyện thứ hai là nguồn của độ sâu. Bốn thể hệ ở mục trước đều cần độ sâu từ đâu đó. Có ba cách lấy nó: một ảnh, hai ảnh, hoặc cảm biến chủ động. Chúng khác nhau không phải ở độ chính xác mà ở chỗ *cái nào xác định được tỉ lệ tuyệt đối*. Cuối cùng, ta đặt SLAM và VIO vào đúng vị trí: không phải một chủ đề riêng mà là hệ thống duy nhất bắt buộc giải cả ba bài toán — biểu diễn, độ sâu, tối ưu — dưới ràng buộc thời gian thực.

Học mục này thì làm được gì mà trước đó không làm được? Ba việc: chọn được biểu diễn cho một pipeline mới mà không phải viết lại nó sáu tháng sau; nhìn một mô-đun độ sâu và nói ngay nó có quyền xuất hiện trong phần dư metric hay không; và đọc được một mô hình học trên point cloud để biết nó mù với cái gì.

TRỰC GIÁC

Chọn biểu diễn 3D giống chọn hệ toạ độ để viết một bài toán cơ học. Bài toán không đổi, nhưng viết trong hệ Descartes thì trọng lực đơn giản còn chuyển động tròn phức tạp; trong hệ cực thì ngược lại. Voxel là hệ Descartes của 3D: đều đặn, dễ lập trình, và lãng phí không gian vì phần lớn không gian là rỗng. Point cloud là “danh sách các chỗ có vật chất”: gọn đúng bằng lượng thông tin thật, nhưng bạn phải tự dựng lại khái niệm hàng xóm mỗi lần cần tới nó.

B. Năm biểu diễn và cái giá của mỗi cái

Năm biểu diễn dưới đây không ngang hàng nhau. Chúng chia làm ba nhóm theo *nguồn gốc*, và biết nhóm nào giúp đoán ngay chỗ hỏng:

- **Biểu diễn đo được** — voxel và point cloud, ra thẳng từ cảm biến. Chỗ hỏng của chúng là chỗ hỏng của cảm biến: mật độ không đều, vùng khuất, nhiễu phụ thuộc khoảng cách.
- **Biểu diễn đồ hoạ** — mesh. Chỗ hỏng là bước tái dựng bề mặt đứng trước nó.
- **Biểu diễn được tối ưu** — **trường ẩn** (*implicit field*) và **Gaussian splatting**. Chúng không tồn tại cho tới khi chạy xong một vòng tối ưu trên chính cảnh đó, nên chỗ hỏng của chúng là chỗ hỏng của hàm mất mát đã sinh ra chúng.

Bảng 1.2. Năm biểu diễn 3D. Cột cuối là cột quyết định: mỗi biểu diễn hỏng ở một chỗ khác nhau, và chỗ đó thường lộ ra muộn.

Biểu diễn	Cơ chế lưu	Được	Mất / hỏng khi
Voxel	Lưới đều 3D, mỗi ô một giá trị	Conv 3D dùng được nguyên xi; lân cận cho sẵn	Bộ nhớ theo lũy thừa ba; hỏng ngay khi cần độ phân giải cao
Point cloud	Tập điểm không thứ tự	Gọn, hợp với LiDAR và stereo	Không có cấu trúc lân cận; phải bất biến hoán vị; mật độ không đều
Mesh	Đỉnh + mặt	Chuẩn của đồ hoạ và mô phỏng; bề mặt tường minh	Tối ưu topology gần như không khả vi; mặt tự cắt nhau
Implicit field (SDF, occupancy)	Một hàm $\mathbb{R}^3 \rightarrow \mathbb{R}$, thường là MLP	Độ phân giải liên tục; bề mặt là tập mức 0, định nghĩa sạch	Render chậm vì phải lấy mẫu nhiều lần dọc tia; khó chỉnh sửa cục bộ
Gaussian splatting	Tập Gaussian dị hướng có màu và độ đục	Rasterize được nên render rất nhanh	Dung lượng lớn; hình học không tường minh nên trích mesh khó

Hình 1.6 vẽ cùng một mặt cong bằng cả năm biểu diễn, đặt cạnh nhau — đây là cách nhanh nhất để thấy mỗi cái giữ lại gì và vứt đi gì.

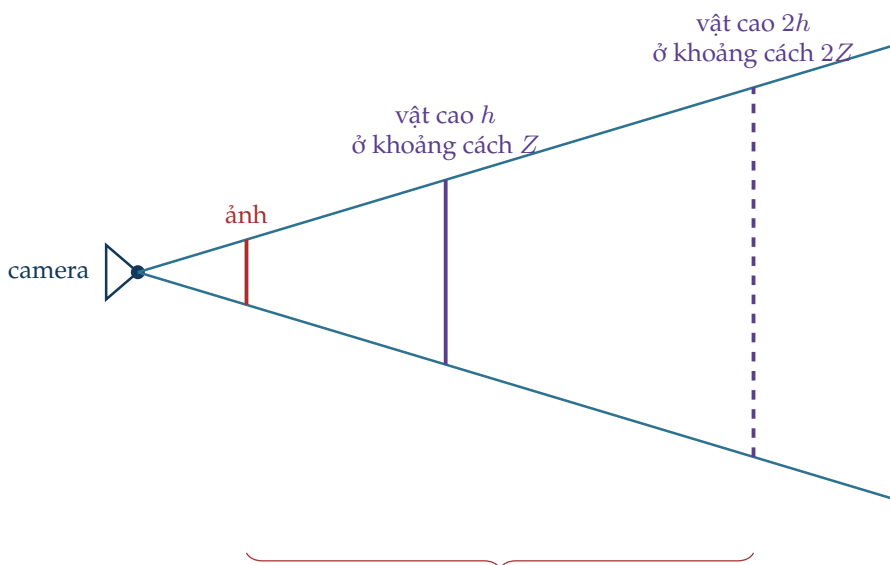


Hình 1.6. Cùng một mặt cong, năm biểu diễn. Nhìn hàng chú thích dưới cùng thì thấy ngay điều mà bảng không nói được: mỗi biểu diễn giữ lại một loại thông tin và vứt đi loại khác. Point cloud biết ở đâu có vật chất nhưng không biết đâu là mặt; voxel biết cả không gian rỗng nhưng trả tiền cho nó; mesh biết mặt nhưng khoá cứng topology; SDF định nghĩa mặt sạch nhất nhưng phải dò dọc tia mới render được; Gaussian render nhanh nhất nhưng không có mặt để mà trích ra.

Con số làm cho dòng đầu bảng thành cụ thể. Một lưới 512^3 chứa khoảng $1,34 \times 10^8$ ô; lưu một số thực 32-bit cho mỗi ô đã tốn 512 MiB, và một tầng conv 3D với 32 kênh ở cùng độ phân giải cần 16 GiB chỉ để giữ *activation*. Nâng lên 1024^3 thì nhân tám. Trong khi đó một quét LiDAR dày đặc của cùng cảnh có chừng 2×10^5 điểm, tức 2,4 MB nếu lưu ba toạ độ float — ít hơn hai bậc độ lớn. Toàn bộ dòng **tích chập thưa** (*sparse convolution*) tồn tại chỉ vì khoảng cách này: hơn 99 % ô của lưới đều là rỗng, nên trả tiền cho chúng là trả tiền cho hư không.

C. Độ sâu đến từ đâu, và cái gì cố định tỉ lệ

Với ảnh đơn, bài toán độ sâu không phải khó — nó *không xác định được*. Nhân toàn bộ cảnh với hệ số s và nhân luôn mọi vị trí camera với s thì mọi ảnh thu được giống hệt nhau. Hình 1.7 là toàn bộ chứng minh, vẽ ra trong một hình.



Hai cảnh khác nhau, **một ảnh duy nhất**. Không phép đo nào trong ảnh phân biệt được chúng.

Hình 1.7. Tính không xác định được của tỉ lệ trong ảnh đơn. Nhân cảnh với s và nhân quãng đường camera với cùng s cho ra đúng bộ ảnh cũ; vì vậy *không lượng dữ liệu nào* chứa được, chỉ prior mới chứa được. Đây cũng chính là lý do SLAM đơn mắt có một bậc tự do tỉ lệ, và vì sao chỉ cần thêm một gia tốc kế — một cảm biến biết đơn vị mét — là bậc tự do đó biến mất.

Đây đúng là hiện tượng **không xác định được** (*non-identifiability*) ở chương 4: dữ liệu không chứa thông tin để phân biệt hai giả thuyết.

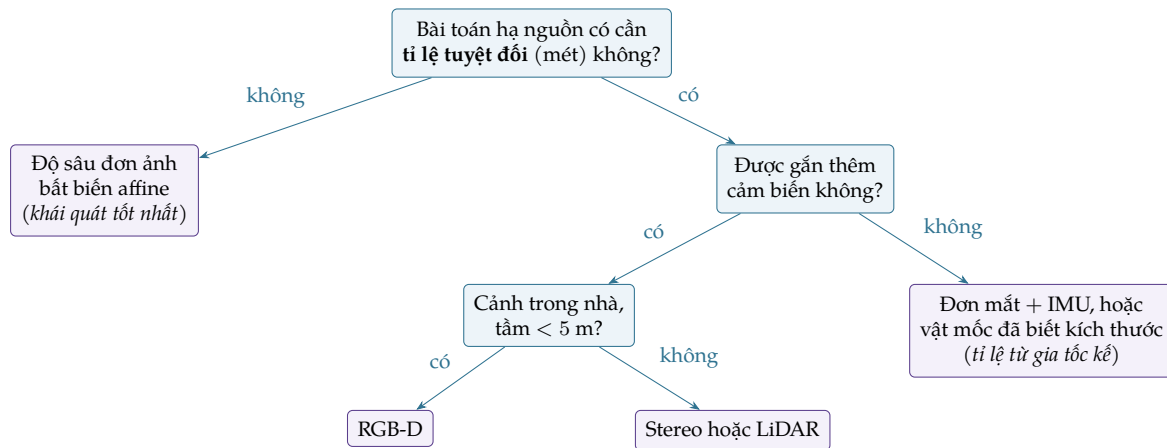
↪ **trường hợp riêng của B/04** — scale ambiguity là một ca sạch của tính không xác định được: hai tham số khác nhau sinh ra cùng một phân phối quan sát, nên thêm dữ liệu không giúp gì.

Một mạng độ sâu đơn ảnh phá thể bề tắc bằng cách học kích thước điển hình của vật thể và mối liên hệ với tiêu cự; nghĩa là con số nó trả về là một *suy đoán có học*, không phải một phép đo. Nó vì thế sai theo kiểu rất đặc trưng ở đúng những cảnh phá vỡ prior kích thước: mô hình thu nhỏ, ảnh chụp từ trên cao, nội soi, ảnh macro.

Bốn nguồn độ sâu, đọc theo cái mà mỗi nguồn dùng để phá thể bề tắc:

- **Đơn ảnh, metric** — phá bằng prior kích thước. Cho ra mét ngay, nhưng chỉ đúng trong miền đã học.
- **Đơn ảnh, bất biến affine** — không phá, mà *bỏ* tỉ lệ đi: huấn luyện trên nhiều tập dữ liệu hỗn tạp với hàm mất mát bất biến với phép biến đổi affine của độ sâu, nên mô hình chỉ phải học thứ tự và hình dạng tương đối [5], [6]. Khái quát rất tốt; đổi lại mỗi khung hình còn dư hai ẩn (tỉ lệ và độ dịch) và hai ẩn đó *không nhất quán giữa các khung hình*.
- **Stereo** — phá bằng vật lý: đường cơ sở đã biết cho tỉ lệ. Cần hiệu chỉnh chính xác, cần texture để khớp, và có chân trời độ sâu tính được từ $\Delta Z = Z^2 \Delta d / (fb)$.
- **RGB-D** — phá bằng cảm biến chủ động: chiếu ánh sáng ra và đo phản hồi, nên có tỉ lệ tuyệt đối ngay cả trên tường trắng. Chết dưới nắng, trên mặt gương, trên vật hấp thụ hồng ngoại; có sai lệch phụ thuộc khoảng cách cần hiệu chuẩn riêng.

Hình 1.8 gói bốn lựa chọn đó thành một cây quyết định dùng được ngay.



Hình 1.8. Cây quyết định chọn nguồn độ sâu. Câu hỏi đầu tiên không phải “mô hình nào chính xác nhất” mà “bài toán của tôi có thật sự cần mét không”. Rất nhiều bài toán chỉ cần độ sâu tương đối: tránh vật cản gần, sắp thứ tự lớp, tách nền. Ở đó lựa chọn bất biến affine vừa rẻ vừa bền hơn hẳn.

D. SLAM và VIO đứng ở đâu

Bạn đọc chương này đã sống trong SLAM nên mục này không nhắc lại cơ chế; nó chỉ đặt SLAM vào bức tranh bốn thể hệ. SLAM là thể hệ một chạy dưới ràng buộc thời gian thực, và toàn bộ kiến trúc *frontend/backend* là hệ quả trực tiếp của ràng buộc đó: frontend phải quyết định tham lam trong vài mili-giây mỗi khung hình nên chỉ dùng ràng buộc cục bộ, còn backend là nơi hình học thực sự được ép đúng trên một cửa sổ hoặc trên toàn đồ thị [7], [8]. **Đóng vòng** (*loop closure*) không phải một tính năng phụ mà là thứ duy nhất đối *bậc* của sai số: không có nó, sai số tích lũy tăng đơn điệu theo quãng đường; có nó, sai số bị chặn bởi độ chính xác của phép đóng vòng.

Bốn thể hệ chạ vào SLAM ở ba chỗ khác nhau:

- **Thể hệ lai thay frontend** — đặc trưng và khớp học được, backend giữ nguyên. Đây là hướng đã thành thực dụng.

- **Thế hệ lai, bản triệt để hơn** — dòng DROID-SLAM dùng **dòng quang** (*optical flow*) dày đặc học được nhưng vẫn nhét một tầng hiệu chỉnh chùm tia khả vi vào giữa, nên nó vẫn là thế hệ hai chứ không phải thế hệ bốn.
- **Thế hệ ba thay bản đồ** — giữ tracker cổ điển nhưng lưu bản đồ bằng trường bức xạ hoặc Gaussian. Đánh đổi bản đồ thừa lấy khả năng render: đáng giá cho robot cần nhìn lại cảnh, đáng ngờ cho robot chỉ cần định vị.

Về đánh giá chỉ cần một câu nhắc: *ATE* cần một phép căn chỉnh toàn cục trước khi đo, vì quỹ đạo ước lượng sống trong một hệ toạ độ tự do — với đơn mắt thì tự do cả tỉ lệ. Chính phép căn chỉnh đó hấp thụ một phần *drift* toàn cục. *RPE* thì đo trên các đoạn ngắn nên không cần căn chỉnh, và vì thế nhạy với lỗi cục bộ mà *ATE* làm phẳng đi [9]. Hai chỉ số phải đọc cùng nhau, và mọi thảo luận sâu hơn về giao thức — chia dữ liệu, số lần lặp, khoảng tin cậy — nằm ở B/07-chia-du-lieu-va-danh-gia.

↔ **ràng buộc bởi** B/07 — phép căn chỉnh của *ATE* là một bước tiền xử lý của giao thức đánh giá, nên nó quyết định lỗi nào còn nhìn thấy được.

E. Học sâu trên point cloud

Muốn đưa mạng nơ-ron lên point cloud thì vấp ngay vào một trở ngại cấu trúc: đầu vào là một *tập*, không phải chuỗi hay lưới. Với n điểm có $n!$ cách đánh số cùng mô tả một hình dạng, nên bất kỳ hàm nào không bất biến với hoán vị đều đang học lại cùng một hình dạng hàng giai thừa lần.

I MẠCH 2 — từ tập điểm tới mạng dùng được

- Vấn đề** đầu vào là tập không thứ tự, mà MLP thường thì không bất biến hoán vị.

Giải pháp ép mọi điểm qua cùng một MLP chia sẻ trọng số rồi gộp bằng một **hàm đối xứng** (*symmetric function*) — lấy max theo từng chiều đặc trưng [10].

Giả định hình dạng mô tả được đủ tốt bằng một vectơ đặc trưng toàn cục.

Hệ quả bất biến hoán vị ở đây là bất biến *theo cấu trúc*, không phải học được từ **tăng cường dữ liệu** (*data augmentation*), nên nó đúng tuyệt đối chứ không đúng gần đúng.
- Vấn đề** max toàn cục vứt bỏ mọi cấu trúc cục bộ: đầu ra chỉ phụ thuộc một tập điểm “tới hạn” rất nhỏ — những điểm đang giữ giá trị lớn nhất ở mỗi chiều.

Giá phải trả thêm hay bớt điểm bên trong bao lồi của tập tới hạn không đổi đầu ra: rất bền với thay đổi mật độ, nhưng mù với chi tiết bề mặt mịn.

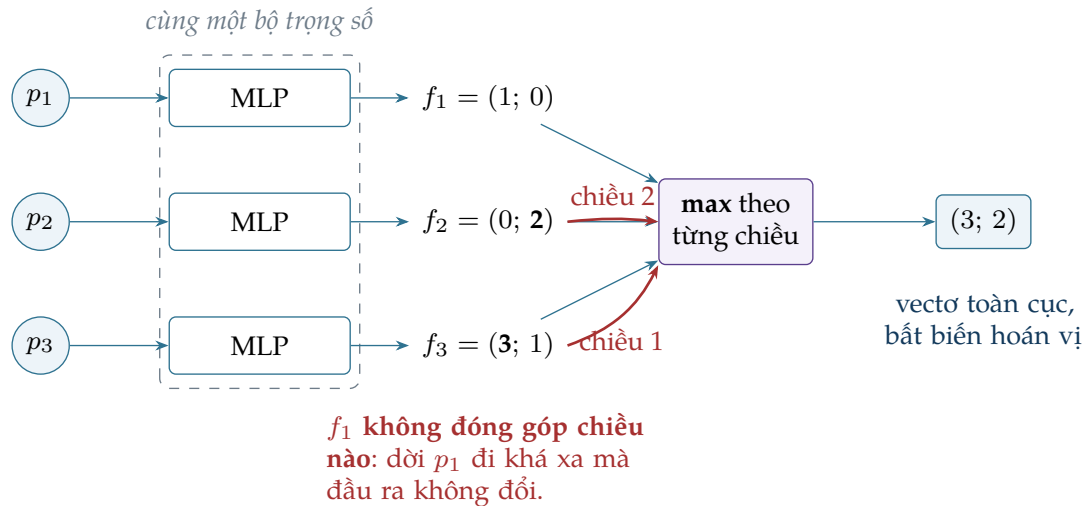
Giải pháp lồng PointNet vào chính nó theo phân cấp không gian — nhóm điểm theo bán kính, chạy PointNet trên từng nhóm, rồi lặp lại ở tỉ lệ lớn hơn [11].

Vấn đề mới chi phí truy vấn lân cận thành nút cổ chai, và kết quả nhạy với việc chọn bán kính.
- Vấn đề** ở quy mô một cảnh đường phố với hàng trăm nghìn điểm, truy vấn lân cận là quá đắt.

Giải pháp quay lại lưới nhưng chỉ giữ ô không rỗng (*sparse convolution*), hoặc gộp điểm theo cột dọc thành *pillar* rồi rơi về một bài toán 2D quen thuộc [12].

Giả định cảnh ngoài trời gần như phẳng theo phương thẳng đứng nên nén trục z mất ít thông tin.

Giá phải trả giả định đó sai ngay trong nhà, nơi vật thể chồng lên nhau theo chiều cao.



Hình 1.9. Hàm đối xứng của PointNet, vẽ trên đúng ba điểm của ví dụ tính tay. Mọi điểm đi qua cùng một MLP rồi được gộp bằng phép max theo từng chiều, nên xáo trộn thứ tự ba điểm cho ra cùng một vector $(3; 2)$ — tính bất biến ở đây là bất biến theo cấu trúc, không phải học được. Nhưng hình cũng phơi ra cái giá: đầu ra chỉ do *tập điểm tới hạn* quyết định (ở đây là f_2 và f_3), còn f_1 hoàn toàn vô hình. Đúng cái mẹo giữ cho mạng bất biến cũng là đúng cái mẹo làm nó mù.

Hình 1.9 vẽ đúng cơ chế đó trên ba điểm.

Tính tay để thấy hàm đối xứng thật sự làm gì. Lấy ba điểm đã qua MLP thành các đặc trưng hai chiều $f_1 = (1; 0)$, $f_2 = (0; 2)$, $f_3 = (3; 1)$. Gộp bằng max theo từng chiều cho $(3; 2)$. Đảo thứ tự ba điểm bằng bất kỳ hoán vị nào vẫn cho $(3; 2)$ — đó là toàn bộ mẹo của PointNet. Nhưng phép tính đó cũng phơi ra ngay giới hạn: chiều thứ nhất chỉ do f_3 đóng góp, chiều thứ hai chỉ do f_2 ; điểm f_1 hoàn toàn không ảnh hưởng, và bạn có thể dời nó đi khá xa mà vector toàn cục không nhúc nhích. *Đúng cái mẹo giữ cho mạng bất biến cũng là đúng cái mẹo làm nó mù.* Cùng cơ chế này sẽ xuất hiện lại ở C/17-mo-hình-sinh/02-toi-uu-tren-pixel, nơi ma trận Gram vút bỏ vị trí không gian để giữ lại “phong cách” — hai kỹ thuật trông không liên quan, một nguyên lý.

↪ cùng nguyên lý với C/17/02 — ma trận Gram gộp trên vị trí đúng như PointNet gộp trên điểm: một hàm đối xứng mua tính bất biến bằng chính thông tin bị gộp đi.

↪ trường hợp riêng của B/10 — bất biến hoán vị là một ví dụ của việc cài thiên lệch quy nạp thẳng vào kiến trúc thay vì học nó từ dữ liệu.

F. Giới hạn và trường hợp hỏng

Điểm yếu chung của cả mục này là các biểu diễn không chuyển đổi qua lại mà không mất mát. Điểm sang mesh cần một bước tái dựng bề mặt vốn tự nó là bài toán ước lượng có prior riêng; mesh sang SDF cần mesh kín, điều gần như không đúng với mesh quét thực tế; Gaussian sang mesh là bài toán mở. Vì vậy lỗi thiết kế đắt nhất trong một pipeline 3D là chọn biểu diễn theo khâu đầu tiên thay vì theo khâu cuối cùng cần dùng kết quả.

Hai chế độ hỏng đáng nhớ riêng:

- **Độ sâu học được:** nguy hiểm nhất không phải sai số lớn mà là sai số *không nhất quán theo thời gian* — mỗi khung hình một tỉ lệ khác nhau trong khi từng khung hình đều trông hợp lý.
- **Point cloud:** thay đổi mật độ giữa lúc huấn luyện và lúc chạy. Mô hình huấn luyện trên LiDAR 64 tia thường tụt mạnh trên 32 tia, và triệu chứng là *mất vật ở xa trước tiên* — đúng nơi mật độ giảm nhanh nhất.

CẠM BẦY

Cạm bẫy quen thuộc với người làm SLAM: lấy độ sâu đơn ảnh làm phép đo trong một bộ ước lượng có trọng số. Nó không những sai tỉ lệ, mà tỉ lệ đó còn *trôi giữa các khung hình*. Sai số của nó vì thế có tương quan mạnh theo thời gian. Đó là vi phạm đúng giả định nhiều trắng độc lập mà bộ lọc dựa vào, và hệ quả là hiệp phương sai bị đánh giá quá lạc quan. Có hai cách dùng an toàn. Cách thứ nhất: coi nó như ràng buộc *tương đối* bên trong một khung hình, ví dụ thứ tự độ sâu hay tính phẳng của mặt. Cách thứ hai: ước lượng tường minh một tỉ lệ và một độ dịch cho mỗi khung hình, rồi đưa hai ẩn đó vào bài toán tối ưu. Cách sai là nhét mét thẳng vào phần dư.

CẦN NHỚ

Biểu diễn quyết định phép toán nào rẻ, và vì thế quyết định bài toán nào bạn còn giải được về sau: hãy chọn nó theo khâu *cuối cùng* của pipeline, không theo khâu đầu tiên. Còn về độ sâu: chỉ có baseline và cảm biến chủ động cho ra mét; mọi thứ khác cho ra một suy đoán có học, và suy đoán đó không được phép xuất hiện trong phần dư metric mà không kèm hai ẩn tỉ lệ và độ dịch. [T4]

G. Kết nối

Mục này cung cấp vốn từ cho [C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat](#), nơi hai biểu diễn cuối bảng được đào sâu như hai câu trả lời cho cùng một bài toán render. Bất biến hoán vị của PointNet là trường hợp riêng của nguyên tắc thiết kế theo bất biến ở B/10-thiet-ke-kien-truc; tính không xác định được của tỉ lệ là trường hợp riêng của B/04-nen-tang-hoc-tu-du-lieu. Phần ATE/RPE nổi thẳng sang B/07-chia-du-lieu-va-danh-gia, còn tụt hiệu năng khi đổi mật độ cảm biến là một ca của B/08-dich-chuyen-phan-phoi.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao bộ nhớ voxel không phải vấn đề kỹ thuật có thể tối ưu đi mà là hệ quả cấu trúc của biểu diễn?
2. Độ sâu đơn ảnh bất biến affine tổng quát tốt hơn độ sâu metric. Cái giá của tính tổng quát đó hiện ra ở đâu khi ghép nhiều khung hình?
3. PointNet bất biến hoán vị “theo cấu trúc”. Vì sao điều đó mạnh hơn hẳn việc học tính bất biến bằng augmentation, và nó khiến mạng mù với cái gì?
4. Nếu loop closure bị tắt, sai số của SLAM đổi bậc như thế nào, và chỉ số nào trong hai chỉ số ATE/RPE phản ánh sự thay đổi đó rõ hơn?
5. Một mô hình phát hiện vật thể trên LiDAR 64 tia được đưa sang cảm biến 32 tia. Bạn dự đoán triệu chứng đầu tiên là gì, và vì sao?
6. Hình 1.7 nói rằng không lượng dữ liệu nào phá được tính không xác định của tỉ lệ. Vậy vì sao một mạng “metric depth” vẫn cho ra mét, và điều đó ngầm giả định gì?

Nguồn

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [2] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [3] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [4] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [5] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [6] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [7] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [8] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [9] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [10] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [11] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [12] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [13] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [14] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [15] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.

1.3 NeRF, 3DGS và hợp nhất cảm biến (Neural Rendering and Sensor Fusion)

PHỤ THUỘC C/16-thi-giac-3d/01-bon-the-he-3d (thế hệ ba nằm ở đầu trong pipeline); C/16-thi-giac-3d/02-bieu-dien-3d-va-depth (trường ẩn và Gaussian trong bảng biểu diễn); A/02-vat-ly-cao-anh (phơi sáng, cân bằng trắng, hiệu chỉnh ngoại).

TÓM TẮT

Mục này giải thích cơ chế thật của **neural rendering**: vì sao NeRF phải biểu diễn cảnh bằng mật độ liên tục chứ không bằng bề mặt, và 3DGS đối cái gì lấy tốc độ. Nửa sau chuyển sang bài toán hợp nhất camera-LiDAR-IMU. Đọc xong bạn quyết định được khi nào một bản đồ render được là đáng giá cho robot, và khi nào bản đồ thừa cổ điển vẫn đúng hơn. Bạn cũng tính được bằng con số vì sao **early fusion** nhạy với lệch hiệu chỉnh còn **late fusion** thì không. Nếu chỉ nhớ một điều: hàm mất mát của neural rendering chỉ ràng buộc *ảnh render*, không ràng buộc *hình học*. Nên PSNR cao không chứng minh bản đồ đúng. Nếu robot sắp va chạm dựa trên bản đồ đó, phải đánh giá bằng độ chính xác hình học so với một quét chân trị.

KIẾN THỨC CẦN CÓ TRƯỚC

- **novel view synthesis (dựng ảnh ở góc nhìn mới)** — bài toán: cho một số ảnh đã chụp của một cảnh, hãy tạo ra ảnh của cảnh đó từ một vị trí camera chưa từng chụp.
- **radiance field (trường bức xạ)** — một hàm gán cho mỗi điểm trong không gian và mỗi hướng nhìn một màu và một độ đặc; đây là “cảnh” mà NeRF học, thay cho bề mặt.
- **volume rendering (dựng ảnh thể tích)** — cách tổng hợp một pixel bằng cách lấy mẫu nhiều điểm dọc theo tia nhìn rồi cộng dồn màu có trọng số theo độ đục tích lũy — thay vì tìm giao điểm với một bề mặt cứng.
- **splat / Gaussian splatting** — “splat” là vết loang một hạt để lại khi bị chiếu xuống mặt ảnh. 3DGS mô tả cảnh bằng hàng triệu hạt Gaussian mờ, và render là chiếu chúng xuống rồi trộn theo thứ tự độ sâu.
- **rasterize** — chuyển một hình học đã chiếu thành pixel bằng cách quét trực tiếp trên lưới ảnh; nhanh hơn nhiều so với đi dò dọc từng tia, và là thứ GPU sinh ra để làm.
- **PSNR** — chỉ số đo sai lệch ảnh dựa trên bình phương trung bình theo pixel, tính bằng decibel; cao là “gần ảnh gốc theo nghĩa số học”, không phải “trông giống theo nghĩa thị giác”.
- **hiệu chỉnh ngoại (extrinsic calibration)** — phép xác định vị trí và hướng tương đối giữa hai cảm biến; sai một độ ở đây biến thành sai vài chục centimét ở tầm hai chục mét.
- **early / late fusion** — hai tầng hợp nhất cảm biến: gộp ở mức đặc trưng (sớm, giàu thông tin) hoặc gộp ở mức kết quả từng nhánh (muộn, bền hơn).

A. Đặt vấn đề

Giả sử SfM đã chạy xong: bạn có pose của bốn mươi tám ảnh và một point cloud thừa. Bây giờ hãy render cảnh đó từ một góc nhìn chưa từng chụp. Kết quả sẽ tệ, và tệ theo cách rất đặc trưng: thủng lỗ ở chỗ không có điểm, viền răng cưa ở biên vật thể, mặt kính và mặt kim loại sai hoàn toàn vì màu của chúng phụ thuộc góc nhìn còn point cloud chỉ lưu được một màu cho mỗi điểm. Vấn đề nằm ở chỗ pipeline cổ điển tối ưu cho *độ chính xác hình học* rồi hy vọng ảnh đẹp theo sau, trong khi cái ta cần ở đây là ngược lại.

Neural rendering đảo ngược thứ tự đó bằng một ý tưởng đơn giản tới mức khó chịu: *nếu thứ ta muốn là ảnh, hãy đặt ảnh vào hàm mất mát*. Tham số hoá cảnh bằng một biểu diễn nào đó, viết một bộ render **khả vi** (*differentiable*) từ biểu diễn đó ra ảnh, rồi hạ **gradient** trên tham số của biểu diễn cho tới khi ảnh render trùng khít mọi ảnh quan sát. Không có bước tìm tương ứng, không tam giác hoá, không ràng buộc epipolar nào được viết ra — hình học nếu có xuất hiện thì xuất hiện như một *tác dụng phụ* của việc khớp ảnh. Câu “nếu có” đó là toàn bộ nội dung mục này.

Học mục này thì làm được gì mà trước đó không làm được? Ba việc rất cụ thể: đánh giá được một đề xuất kiểu “thay bản đồ SLAM bằng Gaussian” thay vì phải tin hay không tin; đọc được một bài neural rendering và chỉ ra ngay giả định nào trong ba giả định nền của nó bị dữ liệu robot vi phạm; và biến câu “hiệu chỉnh phải chính xác” thành một ngân sách sai số tính bằng centimét ở một tầm xa cụ thể.

TRỰC GIÁC

Vì sao NeRF dựng khối chứ không dựng tường? Vì đạo hàm. Nếu cảnh gồm các bề mặt cứng, dịch một bề mặt đi một phần nghìn mét thì một pixel hoặc đổi màu hẳn, hoặc không đổi gì cả. Hàm mất mát khi đó là bậc thang: gradient bằng không gần như mọi nơi, và **hạ gradient** (*gradient descent*) không biết vận về hướng nào. Thay bề mặt cứng bằng một trường mật độ liên tục thì mỗi pixel trở thành tổng có trọng số của hàng chục điểm dọc theo tia, và *mọi* điểm đều nhận được một phần gradient. Đây là phép đổi một công tắc bật/tắt thành một chiết áp: chỉ chiết áp mới nói được nên vận theo chiều nào.

B. NeRF: cảnh là một hàm, ảnh là tích phân dọc tia

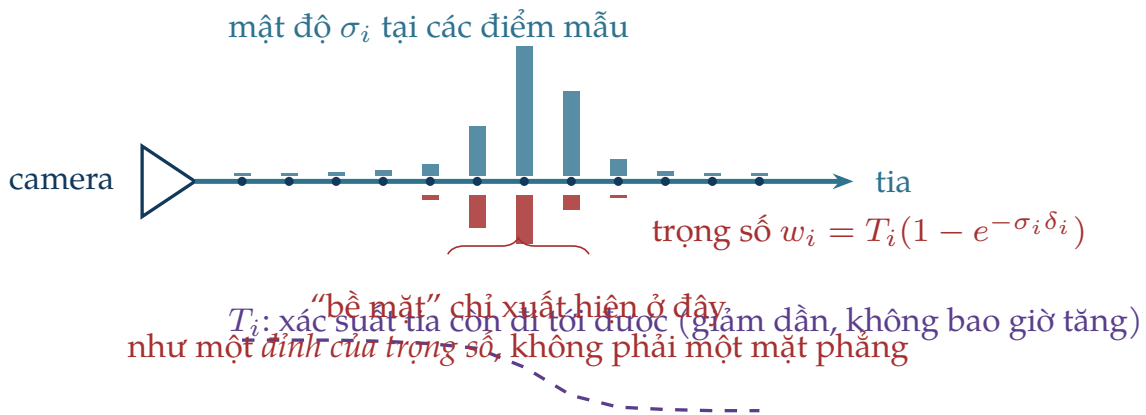
NeRF biểu diễn cảnh bằng một MLP nhận toạ độ 3D và hướng nhìn, trả về mật độ σ và màu c [3]. Để render một pixel, lấy mẫu N điểm dọc tia đi qua pixel đó rồi tổng hợp theo công thức dựng ảnh thể tích rồi raster hoá:

$$C(\mathbf{r}) = \sum_{i=1}^N T_i (1 - e^{-\sigma_i \delta_i}) c_i, \quad T_i = \exp\left(-\sum_{j<i} \sigma_j \delta_j\right).$$

Công thức này thực sự nói gì: T_i là xác suất tia đi tới được điểm thứ i mà chưa bị chặn, còn $1 - e^{-\sigma_i \delta_i}$ là xác suất nó dừng lại đúng ở đó; vậy màu pixel là kỳ vọng của màu tại điểm tia dừng lại. Điều đáng nhớ là mọi số hạng đều khả vi theo σ và c , nên gradient của sai số ảnh chảy ngược được về mọi điểm mẫu — đó là toàn bộ lý do NeRF hoạt động; phần MLP chỉ là một lựa chọn triển khai, và mục sau sẽ thay nó.

→ **tiền đề** C/12/01 — toàn bộ cơ chế đứng trên một điều duy nhất: mọi nút trên đường từ tham số tới ảnh phải khả vi.

Hình 1.10 vẽ đúng ba đại lượng trong công thức trên một tia duy nhất.

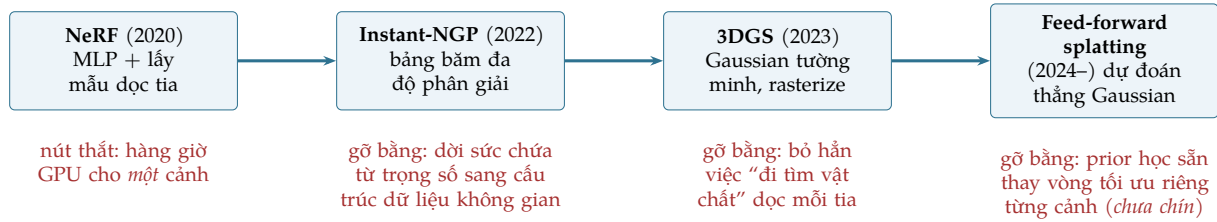


Hình 1.10. Dựng ảnh thể tích, vẽ trên một tia. Mạng không trả lời câu hỏi “mặt ở đâu” mà trả lời “mật độ tại điểm này bao nhiêu” ở hàng chục điểm dọc tia; màu pixel là tổng có trọng số của màu tại các điểm đó. Ba hàng trong hình giải thích công thức: mật độ σ_i quyết định tia dừng lại bao nhiêu ở mỗi điểm, T_i là phần tia còn sống sót tới đó, và tích của hai thứ cho trọng số. Vì mọi số hạng đều liên tục, *mọi* điểm mẫu đều nhận được gradient — đó chính là điều mà một bề mặt cứng không làm được.

Một chi tiết dễ bỏ qua nhưng quyết định chất lượng là **mã hoá vị trí** (*positional encoding*): nhồi thẳng toạ độ vào MLP cho ra ảnh mờ nhoè, vì MLP có thiên lệch mạnh về hàm tần số thấp. Ánh xạ toạ độ qua một dãy sin/cos nhiều tần số trước khi đưa vào mạng là cách trả lại cho mạng khả năng biểu diễn chi tiết sắc nét — cùng câu chuyện *spectral bias* đã gặp ở B/10-thiet-ke-kien-truc.

→ **hệ quả của** B/10 — mã hoá vị trí là cách bù cho một thiên lệch quy nạp có sẵn của MLP, chứ không phải một mẹo kỹ thuật rời rạc.

Cái giá là chi phí tính toán, và nó lớn hơn trực giác nhiều. Một ảnh 800×600 có 480 000 tia; với 64 điểm mẫu thô cộng 128 điểm mẫu tinh, ta cần khoảng $9,2 \times 10^7$ lần chạy MLP cho *một* khung hình. Nhân với hàng chục nghìn bước huấn luyện thì thành nhiều giờ GPU cho một cảnh duy nhất — và đây là nút thắt sinh ra cả dòng công trình tiếp theo, tóm tắt ở Hình 1.11.



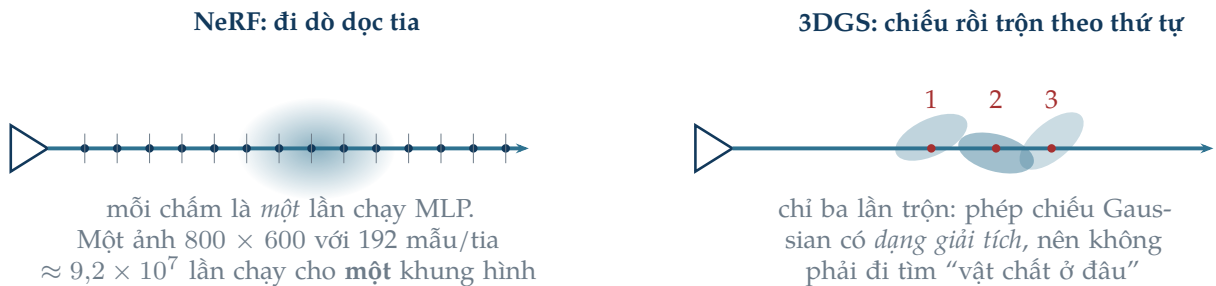
Hình 1.11. Dòng thời gian của neural rendering, đọc theo nút thắt. Mỗi bước không phải “phiên bản tốt hơn” mà là câu trả lời cho đúng một nút cổ chai của bước trước; biết nút thắt nào đang cản trong bài toán của mình thì biết nên dừng ở mốc nào.

Instant-NGP tấn công đúng chi phí đó. Nó chuyển phần lớn sức chứa của mô hình từ trọng số MLP sang một bảng băm đa độ phân giải đặt trong không gian, để MLP chỉ còn nhỏ xíu [13]. Thời gian huấn luyện tụt từ hàng giờ xuống hàng chục giây. Đây là một mẫu hình sẽ gặp lại nhiều lần: khi mạng phải học một hàm có cấu trúc không gian rõ ràng, đưa cấu trúc đó vào *cấu trúc dữ liệu* rẻ hơn nhiều so với bất trọng số học lại nó.

C. 3DGS: đối biểu diễn ẩn lấy biểu diễn tường minh

Ngay cả với bảng băm, NeRF vẫn phải đi dọc từng tia và lấy mẫu — vẫn “đi tìm vật chất ở đâu” cho mỗi pixel mỗi khung hình. 3D Gaussian splatting bỏ hẳn bước tìm kiếm đó: cảnh là một tập hữu hạn Gaussian dị hướng, mỗi cái có vị trí, ma trận hiệp phương sai, độ đục và màu phụ thuộc góc nhìn; render là chiếu từng Gaussian xuống mặt phẳng ảnh, sắp xếp theo độ sâu, rồi trộn theo thứ tự [14]. Toán tổng hợp màu vẫn đúng công thức trên, nhưng phép chiếu có dạng giải tích nên bộ rasterize chạy được ở tốc độ thời gian thực.

Hình 1.12 đặt hai cách tính cùng một pixel cạnh nhau, và cho thấy khoản tiết kiệm đến từ đâu.



Hình 1.12. Cùng một tia, hai cách tính màu pixel. Bên trái, NeRF phải lấy mẫu dày dọc tia vì nó *không biết* trước vật chất nằm ở đâu — toàn bộ chi phí đến từ việc đi tìm. Bên phải, 3DGS đã lưu sẵn vật chất thành các nguyên thể tường minh, nên chỉ cần chiếu chúng xuống mặt ảnh, sắp xếp theo độ sâu và trộn ba lần. Toán tổng hợp màu ở hai bên là một; thứ thay đổi là ai đi tìm — và đó là toàn bộ khoản chênh hai tới ba bậc độ lớn về tốc độ render.

Bảng 1.3. Ba mốc của neural rendering. Cả ba chia chung một tập giả định — pose đã biết, cảnh tĩnh, phơi sáng nhất quán — nên hồng chung ở những chỗ đó, và hồng riêng ở cột cuối.

Phương pháp	Cơ chế	Chi phí	Hồng khi / mất gì
NeRF (MLP)	MLP + lấy mẫu dọc tia + tích phân thể tích	Huấn luyện hàng giờ; render < 1 fps; mô hình vài MB	Chậm cả hai đầu; không chỉnh sửa được cục bộ
Instant-NGP	Bảng băm đa độ phân giải thay phần lớn trọng số	Huấn luyện hàng chục giây; tốn VRAM	Va chạm băm gây nhiễu ở vùng chi tiết
3D Gaussian splatting	Tập Gaussian tường minh, rasterize có sắp xếp	Render thời gian thực; mô hình vài trăm MB	Dung lượng lớn; bề mặt không tường minh; nhạy với nhiễu tạo từ SfM

Tính tay dung lượng một lần cho nhớ. Mỗi Gaussian lưu 3 số vị trí, 3 số tỉ lệ, 4 số quaternion, 1 số độ đục và 48 hệ số **hàm điều hoà cầu** (*spherical harmonics*) bậc ba cho màu: tổng 59 số thực 32-bit, tức 236 byte. Một cảnh phòng khách dựng tốt thường cần cỡ một triệu Gaussian, tức khoảng 236 MB — so với vài megabyte cho trọng số của một NeRF MLP. Đó chính là “giá phải trả” hay được nhắc chung chung: đổi khoảng hai bậc độ lớn dung lượng lấy khoảng hai tới ba bậc độ lớn tốc độ render.

Cái mất thứ hai tinh tế hơn và quan trọng hơn với người làm robot: **hình học kém tường minh**. Trong một trường *SDF*, bề mặt là tập mức 0 — định nghĩa sạch, trích mesh bằng *marching cubes* là chuyện cơ học. Trong 3DGS, “bề mặt” không được định nghĩa ở đâu cả: tối ưu chỉ đòi ảnh render đúng, nên nó thoải mái đặt các Gaussian dẹt lơ lửng trước một bức tường nếu làm vậy khớp ảnh tốt hơn. Muốn trích mesh dùng được, người ta phải thêm ràng buộc ép Gaussian bám bề mặt (dẹt hoá, gấn pháp tuyến) — tức là trả lại đúng phần ràng buộc hình học mà bước đầu đã cố tình bỏ đi.

D. Ba giả định chung, và điều gì xảy ra khi chúng sai

Cả họ neural rendering đứng trên ba giả định, và với dữ liệu quay bằng camera robot thì cả ba đều dễ vỡ:

- **Pose đã biết và chính xác.** Sai lệch vài phần mười độ làm các ảnh mâu thuẫn nhau; vì tối ưu luôn tìm cách giảm loss, nó bù bằng cách sinh ra vật chất giả lơ lửng — hiện tượng *floater*. Đây là lý do 3DGS “nhạy với nhiễu tạo từ SfM”: rác vào thì rác ra, chỉ khác là rác ra trông rất đẹp.
- **Cảnh tĩnh.** Người đi qua khung hình để lại vệt mờ hoặc khối đục lơ lửng ở giữa phòng.
- **Quá trình tạo ảnh nhất quán.** Camera bật phơi sáng và cân bằng trắng tự động vì phạm giả định này liên tục, và mô hình buộc phải giải thích thay đổi độ sáng bằng cách nhét thêm hình học sai. **Ảnh mờ do chuyển động** (*motion blur*) cũng thuộc nhóm này: một khung hình mờ không phải quan sát nhiều của cảnh tĩnh mà là tích phân của cảnh dọc một quãng chuyển động, và mô hình render không có chỗ nào biểu diễn điều đó.

CẠM BÃY

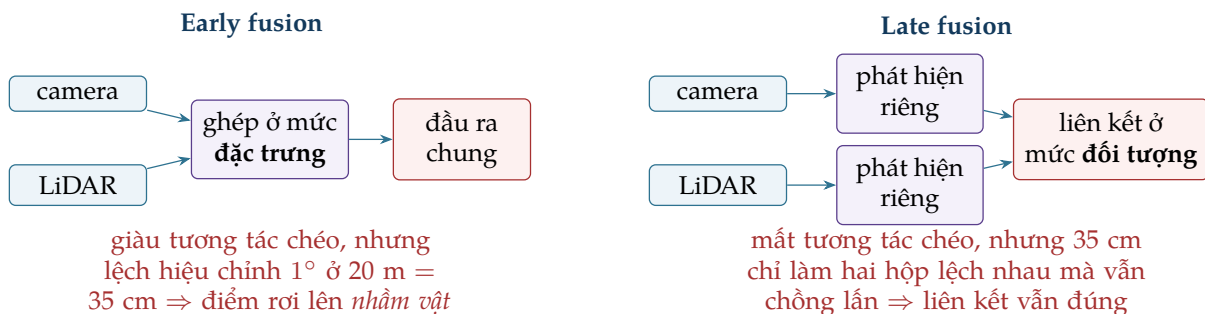
PSNR cao trên các view giữ lại *không* chứng minh hình học đúng. Hàm mất mát chỉ ràng buộc ảnh render; hai cấu hình mật độ rất khác nhau có thể sinh ra cùng bộ ảnh, đặc biệt khi màu được phép phụ thuộc hướng nhìn — mô hình có thể “vẽ” thứ đáng lẽ phải “dựng”. Triệu chứng chẩn đoán: chất lượng đẹp ở các view nội suy gần quỹ đạo chụp và sập ngay khi ngoại suy ra ngoài. Nếu bạn định dùng đầu ra làm bản đồ để robot va chạm hay bám bề mặt, hãy đánh giá bằng độ chính xác hình học so với một quét chân trị, không phải bằng PSNR hay *LPIPS* [15].

Với người làm SLAM, hệ quả thực hành gọn lại thành một câu hỏi: *bạn có thực sự cần render không?* Nếu chỉ cần định vị, bản đồ thừa của SLAM cổ điển vẫn là lựa chọn đúng — rẻ hơn nhiều bậc và có sai số kiểm chứng được. Neural rendering đáng đưa vào khi bài toán hạ nguồn thật sự cần ảnh: kiểm tra trực quan từ xa, sinh dữ liệu huấn luyện từ góc nhìn mới, mô phỏng cảm biến, hoặc định vị lại bằng cách so ảnh render với ảnh thật.

→ cùng nguyên lý với C/17/02 — cùng một ý tưởng “đặt thứ ta muốn vào hàm mất mát rồi tối ưu trên chính đối tượng cần tạo ra”, chỉ khác biến tối ưu là ảnh thay vì cảnh.

E. Hợp nhất camera, LiDAR và IMU

Ba cảm biến này bù trừ cho nhau gần như hoàn hảo: camera cho ngữ nghĩa và độ phân giải góc cao nhưng không có tỉ lệ; LiDAR cho hình học metric thừa và bền với ánh sáng; IMU cho chuyển động tần số cao và quan sát được phương trọng lực nhưng trôi rất nhanh khi tích phân. Câu hỏi thiết kế là hợp nhất ở tầng nào (Hình 1.13).



Hình 1.13. Cùng một sai số hiệu chỉnh, hai hậu quả khác hẳn nhau. Early fusion liên kết ở mức pixel nên sai số biến thành *đầu vào rác*; late fusion liên kết ở mức đối tượng, nơi vài chục centimet vẫn nằm trong kích thước của vật, nên sai số chỉ thành nhiễu đầu ra. Đó là toàn bộ nội dung của cuộc đánh đổi “giàu thông tin” đổi lại “bền vững”.

Con số trong hình tính ra như sau: sai số hiệu chỉnh ngoại xoay 1° chuyển thành sai số ngang $\tan 1^\circ \approx 17,5$ mm trên mỗi mét tầm xa, nên ở 20 m là 35 cm. Lệch đồng bộ thời gian sinh ra đúng loại lỗi đó, nhưng âm thầm hơn vì nó chỉ xuất hiện khi có chuyển động. Robot quay 100°/s với lệch nhãn thời gian 5 ms cho sai lệch góc 0,5°. Con số này tỉ lệ thuận với tốc độ quay. Nghĩa là hệ thống chạy hoàn hảo khi đứng yên, rồi hỏng đúng lúc đang xoay nhanh — đúng lúc bạn cần nó nhất. Đây là lý do một nhận định lặp đi lặp lại trong thực hành: ở các hệ đa cảm biến, phần lớn thời gian sửa lỗi nằm ở đồng bộ thời gian và hiệu chỉnh ngoại chứ không ở thuật toán hợp nhất.

CẦN NHỚ

Sai số hiệu chỉnh và sai số đồng bộ không phải nhiều — chúng là sai số *có hệ thống*, phụ thuộc tầm xa và tốc độ, nên không trung bình về không khi gom nhiều quan sát. Hệ quả bắt buộc: chúng phải được đưa vào mô hình như biến cần ước lượng (hiệu chỉnh trực tuyến, ước lượng độ trễ t_d) chứ không phải được hy vọng là nhỏ. Và với neural rendering, điều tương đương là: hàm mất mát ràng buộc ảnh, nên hình học phải được kiểm tra bằng một phép đo bên ngoài. [T4]

F. Giới hạn và trường hợp hỏng

Giới hạn chung của cả họ neural rendering là tính riêng cho từng cảnh: một mô hình đã tối ưu xong không dùng lại được cho cảnh khác, nên chi phí là chi phí *lặp lại* chứ không phải chi phí một lần như huấn luyện một *backbone*. Các dòng công trình lai — mô hình tiến thẳng dự đoán thẳng tập Gaussian — đang xoá dần ranh giới này, nhưng tại thời điểm viết, chất lượng của chúng vẫn dưới bản tối ưu riêng từng cảnh; đây là quan sát về xu hướng, chưa phải một kết quả đã ổn định.

Về hợp nhất, chế độ hỏng đáng sợ nhất là hỏng *im lặng* của một nhánh. Mưa làm LiDAR sinh điểm giả; đường hầm làm camera mất hết đặc trưng; rung động làm IMU bão hoà. Một hệ hợp nhất không kiểm tra tính nhất quán giữa các nhánh sẽ vẫn cho ra ước lượng trông bình thường, với hiệp phương sai cũng trông bình thường. Vì vậy phần đáng đầu tư trong một hệ thật thường không phải bộ hợp nhất mà là bộ chẩn đoán từng nhánh: phần dư theo từng cảm biến, kiểm tra chi bình phương, và một cơ chế hạ cấp có kiểm soát khi một nhánh bị loại.

↔ **hệ quả của E/25** — hỏng im lặng của một nhánh cảm biến là trường hợp cụ thể của yêu cầu chung: hệ thống phải biết lúc nào nó không đáng tin.

G. Kết nối

Mục này khép lại chương 16 và nối ngược lên [C/16-thi-giac-3d/01-bon-the-he-3d](#): neural rendering là thể hệ ba, đứng *sau* khâu pose, nên nó tiêu thụ đầu ra của thể hệ một chứ không thay thế. Ý tưởng “đặt cái ta muốn vào hàm mất mát rồi hạ gradient trên chính đối tượng cần tạo ra” sẽ trở lại nguyên vẹn ở [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#), chỉ khác là biến tối ưu ở đó là ảnh chứ không phải cảnh. Các giả định về quá trình tạo ảnh thuộc [A/02-vat-ly-tao-anh](#); việc chẩn đoán từng nhánh cảm biến và hạ cấp có kiểm soát thuộc [E/25-do-tin-cay-va-van-hanh](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao biểu diễn cảnh bằng mật độ liên tục là điều kiện cần để gradient descent hoạt động, trong khi bề mặt cứng thì không?
2. 3DGS nhanh hơn NeRF nhờ đổi cái gì lấy cái gì, và vì sao cái mất đó lại đặc biệt bất tiện cho robot?
3. Một cảnh có PSNR rất cao trên các view giữ lại nhưng bản đồ hình học sai. Cơ chế nào cho phép hai điều đó cùng đúng, và bạn thiết kế phép thử nào để phát hiện?
4. Vì sao một hệ early fusion có thể chạy hoàn hảo khi robot đứng yên và hỏng khi robot xoay nhanh, dù không tham số nào thay đổi?
5. Nếu buộc phải chọn giữa một tuần cải tiến thuật toán hợp nhất và một tuần dựng lại quy trình hiệu chỉnh cùng đồng bộ thời gian, bạn chọn cái nào và dựa trên bằng chứng gì?
6. Hình 1.11 đọc theo nút thắt. Trong bài toán của bạn, nút thắt đang cản là thời gian dựng, tốc độ render, hay dung lượng bản đồ — và điều đó chỉ về mốc nào?

Nguồn

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [2] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [3] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [4] S. Wang, V. Leroy, Y. Cabon, et al., “DUSt3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [5] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [6] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [7] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [8] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [9] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [10] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [11] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [12] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [13] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [14] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [15] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.

1.4 Thực hành

Chương này là chương dễ thực hành nhất của Phần C, vì hạ tầng mã nguồn mở đã chín và một chiếc điện thoại là đủ để tạo dữ liệu. Một lưu ý về bản rã trước khi vào danh sách: danh sách dưới đây chốt tại **tháng 9/2026**, và các tên mô hình dẫn đầu bảng sẽ cũ trong 6–12 tháng. Các repo hạ tầng thì bền hơn nhiều: COLMAP, Open3D, evo, gsplat. Hãy đầu tư kỹ năng vào nhóm sau.

Repo và tài nguyên. SfM tham chiếu: [colmap/colmap](#) (giấy phép BSD; lưu ý bản 4.x đổi backend đọc ảnh nên không tái lập bit-for-bit với 3.x) [2]. *Feed-forward geometry*:

facebookresearch/vgg (CVPR 2025 best paper; nhiều pipeline không cần COLMAP dựng trên nó), DUS3R/MASt3R [4]. *Splatting và rendering*: [nerfstudio-project/gspat](https://nerfstudio-project.github.io/) (Apache 2.0, mặc định hợp lý cho nghiên cứu), [graphdeco-inria/gaussian-splatting](https://github.com/graphdeco-inria/gaussian-splatting) (bản gốc, giấy phép hạn chế nhất — đọc kỹ trước khi dùng cho sản phẩm), [nerfstudio-project/nerfstudio](https://nerfstudio-project.github.io/), NVIDIA vkSplatting. *Độ sâu đơn ảnh*: Depth Anything V2, Apple MoGe. *SLAM*: ORB-SLAM3 [8], DROID-SLAM; đánh giá bằng [MichaelGrupp/evo](https://michaelgrupp.github.io/evo/). *Point cloud*: is1-org/Open3D, open-mmlab/mmdetection3d.

Câu hỏi kiểm tra hiểu. Vì sao ATE cần bước căn chỉnh còn RPE thì không, và mỗi cái nhạy với loại lỗi nào? 3DGS nhanh hơn NeRF nhờ đổi cái gì lấy cái gì? Feed-forward geometry tốt hơn COLMAP ở tình huống nào và tệ hơn ở tình huống nào — và bạn nhận ra mình đang ở tình huống nào bằng dấu hiệu gì *trước khi* chạy?

Mini project (vài giờ đến hai ngày). Quay một video 60 giây quanh một vật thể. Chạy hai đường: (a) COLMAP → gspat, (b) một mô hình tiến thẳng → gspat. So ba đại lượng: thời gian tường, PSNR trên các view giữ lại, và độ lệch pose giữa hai đường. Giá trị của bài này không nằm ở con số mà ở chỗ bạn buộc phải quyết định *đường nào là tham chiếu* — và phát hiện ra rằng câu hỏi đó không có câu trả lời hiển nhiên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• SfM	Quy trình cổ điển suy đồng thời pose camera và cấu trúc 3D thừa từ một tập ảnh: tìm tương ứng, ước lượng pose, tam giác hoá, tinh chỉnh. Nó là thể hệ một và vẫn là chuẩn đối chiếu của mọi phương pháp mới.
••• Hiệu chỉnh chùm tia	Bài toán bình phương tối thiểu phi tuyến, tinh chỉnh đồng thời mọi pose và mọi điểm 3D sao cho sai số chiếu lại nhỏ nhất. Điều đáng nhớ nhất: nó trả về phần dư và hiệp phương sai, nên hệ thống tự biết lúc nào nó sai.
•• Point map	Ảnh mà mỗi pixel chứa một toạ độ 3D thay vì một màu. Đây là dạng đầu ra của các mô hình tiến thẳng như DUS3R hay VGGT, thay cho đám mây điểm rời rạc.
••• Vì sao mô hình tiến thẳng hồng “im lặng”?	Vì nó là một bài hồi quy, không phải một bài ước lượng có ràng buộc. Không còn phần dư hình học nào để mâu thuẫn với kết quả, nên cảnh trong hay ngoài phân phối nó đều trả lời với cùng vẻ tự tin.
•• Thị sai	Độ lệch ngang của cùng một điểm giữa ảnh trái và ảnh phải trong hệ stereo. Nó tỉ lệ nghịch với độ sâu: $Z = fb/d$.
•• Vì sao sai số độ sâu stereo tăng theo Z^2 ?	Vì $Z = fb/d$ nên $\Delta Z = Z^2 \Delta d / (fb)$. Hệ quả thực hành: mọi hệ stereo có một “chân trời”, và cách duy nhất đẩy nó ra xa là tăng tiêu cự hoặc đường cơ sở.
••• Scale ambiguity	Phóng to cả cảnh lẫn quãng đường camera cùng một hệ số cho ra đúng bộ ảnh cũ. Vì vậy tỉ lệ tuyệt đối không suy ra được từ ảnh, dù thêm bao nhiêu dữ liệu; chỉ prior hoặc một cảm biến biết mét mới phá được.
•• Voxel	Ô lập phương trong một lưới đều 3D. Ưu điểm là lân cận cho sẵn nên conv 3D dùng được ngay; nhược điểm là số ô tăng theo luỹ thừa ba, và hơn 99% ô là rỗng.
•• SDF	Hàm gán cho mỗi điểm khoảng cách có dấu tới bề mặt gần nhất — âm ở trong, dương ở ngoài. Bề mặt là tập điểm bằng 0, nên trích lưới tam giác là chuyện cơ học; đổi lại render phải dò dọc tia.
•• Point cloud	Tập toạ độ 3D không có thứ tự và không có kết nối. Gọn và hợp với LiDAR, nhưng không có sẵn khái niệm lân cận, nên mọi phép cục bộ đều phải trả tiền tìm kiếm.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● PointNet	Mạng cho tập điểm: mọi điểm qua cùng một MLP rồi gộp bằng phép max theo từng chiều. Phép max không quan tâm thứ tự nên tính bất biến hoán vị là đúng tuyệt đối, không phải học được.
● Bản đồ dựng ra trông đẹp nhưng mọi khoảng cách sai cùng một hệ số — nguyên nhân?	Scale ambiguity chưa được phá: pipeline đơn mắt không có cảm biến nào biết đơn vị mét. Phép phân biệt với lỗi hiệu chỉnh: đo vài vật ở các tầm khác nhau — sai một hệ số <i>không đổi</i> là lỗi tỉ lệ, sai tăng dần theo tầm là lỗi hiệu chỉnh.
● Mô hình LiDAR 64 tia tụt mạnh trên 32 tia, mất vật ở xa trước — nguyên nhân?	Mật độ điểm lúc chạy khác lúc huấn luyện, và vùng xa là nơi mật độ giảm nhanh nhất. Phân biệt với lỗi hiệu chỉnh: lỗi hiệu chỉnh làm lệch vị trí hộp ở mọi tầm, không làm mất hẳn vật ở xa.
●●● Volume rendering	Cách tính màu một pixel bằng cách lấy mẫu nhiều điểm dọc tia rồi cộng dồn màu có trọng số theo độ đục tích lũy. Nó thay câu hỏi “bề mặt ở đâu” bằng câu hỏi “mật độ tại điểm này bao nhiêu”.
●●● Vì sao NeRF dùng mật độ liên tục, không dùng bề mặt?	Vì đạo hàm. Bề mặt cứng làm hàm mất mát thành bậc thang, gradient bằng không gần như mọi nơi. Mật độ liên tục biến mỗi pixel thành tổng có trọng số, nên mọi điểm mẫu đều nhận được gradient.
●● Positional encoding	Ảnh xạ toạ độ qua một dãy sin/cos nhiều tần số trước khi đưa vào MLP. Không có nó thì ảnh mờ nhoè, vì MLP thiên lệch mạnh về hàm tần số thấp.
●●● 3D Gaussian splatting	Biểu diễn cảnh bằng hàng triệu Gaussian dị hướng có màu và độ đục, render bằng cách chiếu rồi trộn theo thứ tự độ sâu. Được tốc độ thời gian thực; mất dung lượng (236 byte mỗi Gaussian) và mất bề mặt tường minh.
●● Vì sao early fusion nhạy với lệch hiệu chỉnh hơn late fusion?	Vì nó liên kết ở mức pixel. Lệch 1° ở 20 m là 35 cm, đủ để điểm LiDAR rơi lên nhầm vật và biến đặc trưng ghép thành rác. Late fusion liên kết ở mức đối tượng, nơi 35 cm vẫn nằm trong kích thước vật.
● PSNR trên view giữ lại rất cao nhưng bản đồ hình học sai — nghĩa là gì?	Mô hình đã “vẽ” thay vì “dựng”: nó khớp ảnh bằng màu phụ thuộc hướng nhìn. Phép phân biệt: ngoại suy ra ngoài quỹ đạo chụp, hoặc so với một quét chân trị.
● Hệ hợp nhất chạy hoàn hảo khi robot đứng yên, hỏng khi xoay nhanh — nguyên nhân?	Lệch đồng bộ thời gian, vì sai số của nó tỉ lệ với tốc độ quay: 5 ms ở $100^\circ/\text{s}$ là $0,5^\circ$. Phân biệt với lệch hiệu chỉnh ngoại: lệch hiệu chỉnh sai cả khi đứng yên.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
COLMAP	Hiện thực tham chiếu của thể hệ một: tìm tương ứng, ước lượng pose, hiệu chỉnh chùm tia. Vẫn là chuẩn để đối chiếu mọi phương pháp mới	Hạ tầng bền; phần lớn dữ liệu huấn luyện của thể hệ bốn cũng do nó tạo ra
ORB-SLAM3	Hiệu chỉnh chùm tia dưới ràng buộc thời gian thực, với tách frontend/backend và đóng vòng để chặn sai số	Chuẩn đối chứng của SLAM đặc trưng thưa
DUST3R / MAST3R / VGGT	Thể hệ bốn: point map trong một hệ toạ độ chung, không cần pose đầu vào	Mạnh ở khởi tạo và ảnh thưa; vẫn thường được nối tiếp bằng một vòng hiệu chỉnh chùm tia
gsplat, nerfstudio	Neural rendering thực dụng: tối ưu biểu diễn cảnh bằng loss render khả vi, tiêu thụ pose từ SfM	Hạ tầng; phần lớn pipeline dựng cảnh hiện nay đi qua đây

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Depth Anything V2, MoGe	Độ sâu đơn ảnh bất biến affine — bỏ tỉ lệ để đổi lấy khả năng khái quát	Dùng làm prior tương đối thì tốt; dùng làm phép đo metric thì phải kèm hai ẩn tỉ lệ và độ dịch
Open3D, mmdetection3d	Học sâu trên point cloud: bất biến hoán vị, phân cấp không gian, tích chập thưa	Hạ tầng cho mảng LiDAR

MINI PROJECT SLAM · Hai đường dựng cảnh 3D và điểm giao của chúng · 1-2 ngày

Mục tiêu	tìm bằng số cái ngưỡng mà ở dưới nó thế hệ tiến thẳng thắn hình học cổ điển, và ở trên nó thì thua. Đo trên chính camera và chính loại cảnh bạn đang chạy SLAM, không phải trên benchmark của paper.
Dữ liệu	một video 60 giây quay quanh một vật thể hoặc một góc phòng, quay bằng đúng camera bạn dùng cho SLAM (giữ nguyên phơi sáng thủ công nếu có). Trích ra bốn tập ảnh với $N = 8, 15, 30, 60$ khung hình phân bố đều, và giữ riêng 10% khung hình làm view kiểm.
Các bước	với mỗi N : (1) chạy đường A — COLMAP → gsplat — và đường B — một mô hình 3D tiến thẳng (DUST3R/MAS3R/VGGT) → gsplat; (2) ghi thời gian tường của từng khâu; (3) ghi PSNR trên view kiểm; (4) ghi độ lệch pose giữa hai đường sau khi căn chỉnh Sim3; (5) ghi lại N nhỏ nhất mà mỗi đường còn khởi tạo được.
Xong khi	bạn vẽ được hai đường PSNR theo N trên cùng một trục và chỉ ra điểm giao bằng con số, kèm bảng thời gian tường và N tối thiểu của từng đường. Câu trả lời phải có dạng “dưới $N = \dots$ đường tiến thẳng thắn, trên đó COLMAP thẳng, và nó tốn thêm ... phút”.
Nối với	C/16/01 giải thích vì sao điểm giao đó <i>phải</i> tồn tại; B/07 cho cách tách view kiểm cho đúng; E/24 cho việc lặp lại nhiều lần trước khi tin một chênh lệch. Bảng số bạn tạo ra ở đây là baseline cho mini project của chương sau.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì phóng to cả cảnh lẫn quãng đường camera cùng một hệ số cho ra chuỗi ảnh y hệt — đây là tính không xác định được, dữ liệu không mang thông tin đó. Chỉ prior kích thước, hoặc một cảm biến biết đơn vị mét (đường cơ sở stereo, LiDAR, gia tốc kế), mới phá được.
- A2. Vì hai bên giải hai bài toán khác nhau. Mô hình tiến thẳng mạnh ở *khởi tạo*: ít ảnh, chồng lấn kém, không pose. Hiệu chỉnh chùm tia mạnh ở *tin chỉnh*. Quan trọng hơn, chỉ nó trả về phần dư và hiệp phương sai, nên chỉ nó cho ta biết lúc nào kết quả không đáng tin.
- A3. Vì đạo hàm. Bề mặt cứng làm hàm mất mát thành bậc thang, gradient bằng không gần như mọi nơi nên không có hướng để đi. Mật độ liên tục biến mỗi pixel thành tổng có trọng số của hàng chục điểm dọc tia, và mọi điểm đều nhận được một phần gradient.
- A4. Vì lệch đồng bộ thời gian chỉ hiện ra khi có chuyển động và tỉ lệ thuận với tốc độ quay: 5 ms ở $100^\circ/\text{s}$ là $0,5^\circ$, ở tầm 20 m thành khoảng 17 cm. Early fusion liên kết ở mức pixel nên độ lệch đó biến điểm LiDAR thành rơi lên nhầm vật, tức là đầu vào rác.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ tới cái bóng của một vật in trên tường. Nhìn cái bóng, bạn không có cách nào biết vật đó to bằng nào và cách tường bao xa: một cái cốc nhỏ để gần và một cái thùng lớn để xa đổ ra đúng cùng một cái bóng. Đó chính xác là chuyện xảy ra khi máy ảnh chụp một cảnh — ảnh là cái bóng, và chiều sâu bị mất khi chụp.

Cả chương này là các cách lấy lại chiều đã mất, và chỉ có ba cách. Cách thứ nhất là đi vòng sang chỗ khác chụp thêm vài cái bóng nữa rồi ghép chúng lại; chỗ nào các cái bóng chỉ về cùng một điểm thì đó là vật thật. Cách này chính xác tới từng centimet, và quan trọng hơn, nếu các cái bóng mâu thuẫn nhau thì bạn biết ngay là mình đã sai ở đâu đó. Cách thứ hai là nặn một mô hình bằng đất

sét trong bóng tối, rồi soi đèn từ đúng những hướng đã chụp và chỉnh đất sét cho tới khi mọi cái bóng khớp. Cách thứ ba là đưa ba tấm ảnh cho một người đã nhìn hàng triệu cảnh và hỏi luôn. Cái giá thì rất rõ. Cách thứ nhất cần bạn đi thêm một vòng và cần cảnh đứng yên. Cách thứ hai cho mô hình đồ bóng đúng ở mọi hướng đã thử. Nhưng nó chưa chắc đúng khi soi từ một hướng mới, vì bạn chỉ ép nó khớp *cái bóng*, không ép nó khớp *hình dạng*. Cách thứ ba trả lời trong một giây và không cần đi đâu cả, nhưng khi gặp một cảnh lạ nó vẫn trả lời tự tin y hệt, và không có cái bóng nào mâu thuẫn để tố cáo nó.

Chỗ không rút gọn được: chỗ nói về sai lệch khi lắp camera và cảm biến với nhau. Cách nói đơn giản tự nhiên là “phải lắp cho thật chuẩn”, nhưng nói vậy là sai trọng tâm: vấn đề không phải sai số lớn hay nhỏ, mà là nó *lắp lại y hệt* ở mọi phép đo, nên gom thêm bao nhiêu phép đo cũng không triệt tiêu được nó. Đây là chỗ phải nói đủ dài mới không sai.

Một câu: *Ảnh làm mất chiều sâu, và bạn chỉ có hai cách lấy lại: đi thêm một vòng để nhìn từ chỗ khác, hoặc đoán bằng kinh nghiệm — chỉ cách thứ nhất mới tự nói cho bạn biết lúc nào nó sai.*

Kết nối

Chương này khép lại phần hình học của Phần C. C/17-mô-hình-sinh lấy lại nguyên vẹn ý tưởng “đặt cái ta muốn vào hàm mất mát rồi tối ưu trên chính đối tượng cần tạo ra”, lần này trên pixel; B/07-chia-du-lieu-va-danh-gia hoàn thiện phần giao thức đánh giá mà chương này chỉ nhắc qua; E/25-do-tin-cay-va-van-hanh biến “cơ chế phát hiện sai từ bên ngoài” thành thiết kế cụ thể.

Tổng kết chương

Ta đã đi từ một bài toán duy nhất — lấy lại chiều đã mất khi chiếu — qua bốn cách giải nó, và thấy chúng không xếp thành một chuỗi tiến bộ mà thành một trục đánh đổi. Ở một đầu trục là hình học cổ điển: giả định nhiều, dữ liệu ít, độ chính xác metric cao, và quan trọng nhất là tự biết lúc nào mình sai. Ở đầu kia là mô hình tiến thẳng: giả định ít, dữ liệu nhiều, chạy được ở chỗ hình học bó tay, và im lặng khi sai. Neural rendering không nằm trên trục đó mà nằm ở một tầng khác — nó không cạnh tranh với SfM mà tiêu thụ đầu ra của SfM, và nó tối ưu cho một mục tiêu khác hẳn là chất lượng ảnh.

Ba chủ đề còn bỏ ngỏ, và cả ba đều là chỗ nên theo dõi trong một hai năm tới: thứ nhất, các mô hình tiến thẳng dự đoán thẳng biểu diễn render được, tức là gộp thể hệ ba và bốn làm một; thứ hai, neural rendering cho cảnh động, nơi giả định cảnh tĩnh bị gỡ bỏ; thứ ba, và thực dụng nhất với robot, cách gắn một ước lượng bất định tin cậy được vào đầu ra của các mô hình học. Chừng nào chưa có nó, thể hệ bốn vẫn chỉ dùng được ở vai trò khởi tạo, không phải vai trò quyết định.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Luận điểm gộp là mỗi lần bỏ một giả định, bạn bù bằng dữ liệu *và* mất một phần khả năng tự chẩn đoán. Theo dấu khoản mất thứ hai qua bốn thể hệ: ở mỗi thể hệ, tín hiệu tự chẩn đoán nào biến mất?
2. Tính không xác định của tỉ lệ ở mục 1 và mục 2, so với việc mô hình ở mục 3 vẫn dựng được cảnh trông rất đúng — hai chỗ này có mâu thuẫn không? Chỉ ra thông tin nào đã lén vào thay cho ràng buộc còn thiếu.
3. Bạn có ba nguồn độ sâu cho cùng một pixel: stereo, độ sâu đơn ảnh học được, và LiDAR thừa. Ba luật sai số ở đây khác hẳn nhau; luật nào khiến việc gán trọng số theo nghịch đảo phương sai trở nên không hợp lệ, và bạn xử lý thế nào?
4. Hình học cổ điển hồng ồn ào còn mô hình học được hồng im lặng. Với một hệ dùng cả hai, bạn dựng cơ chế nào để buộc phần im lặng phát ra tiếng, và cơ chế đó tiêu tốn cái gì?

5. Loop closure ở mục 2 và tinh chỉnh toàn cục sau khâu pose ở mục 1 đều là ràng buộc toàn cục. Vì sao neural rendering ở mục 3 hầu như không có thứ tương đương, và hệ quả với một bản đồ dựng suốt nhiều giờ là gì?
6. Bạn nghi hệ hợp nhất hỏng vì hiệu chỉnh ngoại tại hoặc lệch đồng bộ thời gian chứ không vì thuật toán. Phép thử nào tách hai nguyên nhân đó, và vì sao chuyển động xoay là điều kiện thử tốt hơn chuyển động thẳng?

Nguồn

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [2] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [3] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [4] S. Wang, V. Leroy, Y. Cabon, et al., “DUSt3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [5] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [6] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [7] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [8] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [9] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [10] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [11] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [12] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.
- [13] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [14] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [15] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.